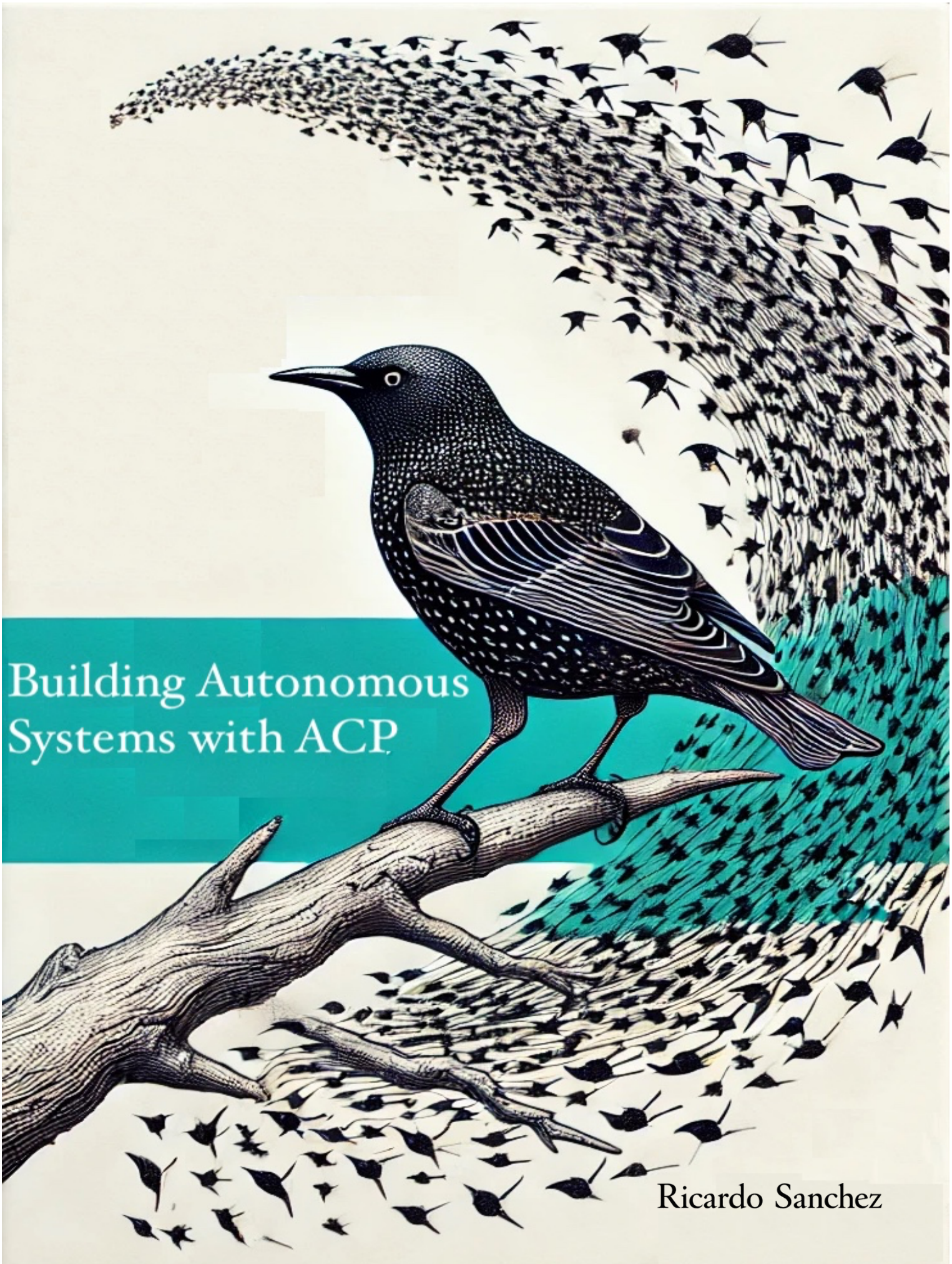

Building Autonomous Systems with ACP
Ricardo Sanchez

Designing Autonomous Systems with the Agent Communication Protocol (ACP)

Learning ACP

Ricardo Sanchez Fuster

Table of Contents

Preface

Designing Autonomous Systems with Agent Communication Protocol

ACP stands for Agent Communication Protocol.

Over the past several decades, computing has undergone several major architectural transitions. Early computing revolved around isolated machines performing well-defined tasks. The rise of the Internet introduced networked services and distributed systems. The cloud era brought elasticity and infrastructure abstraction.

We are now entering another transition: the rise of **autonomous systems**.

Autonomous systems differ from traditional software components in an important way. Instead of simply responding to requests, they **initiate actions, make decisions, and coordinate with other systems**. They operate in dynamic, distributed environments that are increasingly independent of direct human supervision.

Examples of such systems are already appearing across many domains:

- AI research assistants coordinating experiments
- financial trading systems negotiating market interactions
- logistics and supply-chain platforms optimising delivery networks
- distributed robotics systems coordinating physical tasks
- software agents managing infrastructure operations

Each of these systems relies on **collaboration between multiple autonomous participants**.

This shift introduces a new class of technical challenges. Traditional distributed systems were designed around the idea of **services**, stable components reachable at known endpoints. Autonomous systems behave differently. They are often dynamic, mobile, and independent. They may appear and disappear, negotiate capabilities, or operate across organisational boundaries.

The infrastructure supporting machine collaboration has not yet fully adapted to this change.

Most machine-to-machine interaction today still relies on familiar mechanisms:

- REST APIs
- webhooks
- message brokers
- custom authentication layers
- manually configured integrations

These approaches have served the industry well for many years. However, they were largely designed for environments where systems were **predictable, stable, and centrally managed**.

Autonomous systems introduce different requirements.

Agents may not know in advance which peers they will interact with. They may need to negotiate interactions dynamically. They may move between environments or communicate across organisational trust boundaries. In such settings, simple endpoint-based communication models become fragile.

Researchers and practitioners increasingly discuss the future of **multi-agent ecosystems**, where large numbers of autonomous systems cooperate to solve complex tasks. If those ecosystems are to function reliably, they need more than transport. They need a shared collaboration layer.

This is the **machine collaboration problem**.

A Design Principle

The Agent Communication Protocol (ACP) is designed for **interaction, not integration**.

Integration connects systems once. Interaction allows systems to collaborate over time, as conditions change.

This distinction becomes increasingly important as systems evolve. Many architectures succeed at initial integration but degrade as systems change independently. The problem is not the connection, but the persistence of collaboration.

A Subtle Tension

Modern systems appear to work.

APIs connect. Messages flow. Systems respond.

But beneath this apparent success lies a fragile reality:

- endpoints change
- assumptions drift
- coordination breaks

These failures rarely appear at the moment of integration. They emerge later.

What This Book Will Do

This book examines the problem of machine collaboration and introduces ACP as a protocol designed to address it.

The chapters that follow move from conceptual foundations to architecture, identity, messaging, discovery, transport, and deployment models.

In the Introduction, we examine why existing architectural assumptions begin to fail as autonomous systems evolve over time. In **Chapter 1**, we establish the conceptual vocabulary required to reason about autonomous systems. In **Chapter 2**, we examine how those concepts are organised into a layered architecture. Subsequent chapters explore identity, message structure, discovery, and transport in detail.

A Reader's Position

You do not need to accept ACP to read this book.

But you should examine whether the problem it describes exists in your systems.

If it does, the rest of this book provides a framework for thinking about it in a different way

An introduction to the Agent Communication Protocol

The Missing Layer in Autonomous Systems

Systems That Work, Until They Don't

Modern distributed systems are remarkably effective at connecting components. Services expose APIs, messages flow through brokers, and systems react to events. From the perspective of initial delivery, the dominant architectural patterns of the last two decades have been successful.

At first, everything appears to work.

Over time, however, systems do not remain static. Services are replaced, infrastructure is reconfigured, teams evolve, and dependencies shift. What was once a stable integration begins to exhibit unexpected behaviour. Small changes propagate across system boundaries. Failures become harder to diagnose, and coordination between components becomes increasingly fragile.

This pattern is not the result of poor engineering. It is a consequence of a deeper mismatch between how systems are designed and how they behave over time.

The problem is not the connection. It is **sustained interaction under change**.

The Architectural Assumptions Behind Modern Systems

Most distributed architectures are built on implicit assumptions. These assumptions are rarely documented because they hold true in many environments.

A system is expected to know where its peers are located. Endpoints are assumed to be stable or at least slowly evolving. Trust is often inferred from infrastructure, network location, certificates, or shared deployment environments. Interactions are treated as discrete events rather than ongoing relationships.

These assumptions are not incorrect. They are simply incomplete.

They originate from a model of computing in which systems are relatively static, centrally managed, and bounded within a single organisational context. Under those conditions, endpoint-based integration and transport-level security are sufficient.

Autonomous systems do not operate under those conditions.

They are mobile. They evolve independently. They interact across organisational boundaries. They are introduced incrementally rather than designed as a whole. In such environments, the assumptions that once simplified system design begin to introduce fragility.

The result is an architectural tension: systems are built using models that assume stability, while the systems themselves exhibit change.

From Integration to Interaction

Traditional distributed systems are optimised for integration. Integration answers a specific question: how can one system call another?

The solution is well understood. Define an endpoint, establish a contract, and exchange messages. This approach works well when interactions are predictable and bounded.

Autonomous systems introduce a different requirement. They must not only connect; they must **interact repeatedly over time**.

Interaction is not a single event. It is a relationship that evolves between participants. It requires continuity of identity, consistency of semantics, resilience to infrastructure change, and the ability to discover and rediscover participants.

Integration solves the first moment of connection. Interaction governs everything that follows.

This distinction is subtle, but it becomes central as systems scale.

As we will see in **Chapter 1: Concepts**, ACP introduces a vocabulary designed specifically for interaction rather than integration.

The Illusion of Low Friction

Modern tooling has made integration remarkably easy. Developers can connect systems with minimal configuration, often within minutes. This reduction in friction has enabled rapid innovation and widespread adoption of distributed architectures.

However, this ease of integration conceals a longer-term cost.

When systems are connected quickly, they often rely on implicit assumptions rather than explicit models. Identity is inferred rather than defined. Discovery is manual rather than dynamic. Trust is tied to infrastructure rather than embedded in the interaction.

These shortcuts are practical at small scale. At larger scale, they accumulate into systemic complexity.

The cost is not visible at the point of integration. It emerges later, when systems must be changed, extended, or governed.

This is not a failure of individual tools. It reflects the fact that existing approaches optimise for **initial connection**, not for **long-term coordination**.

The Missing Layer

The evolution of distributed computing can be understood in terms of layers.

Networking protocols, such as TCP/IP, enable connectivity between machines. Application protocols such as HTTP enable interaction between services. Messaging systems introduced asynchronous communication patterns.

Each of these layers addressed a specific class of problems.

What remains absent is a layer that defines how **autonomous participants interact as independent entities**.

Specifically, there is no widely adopted standard that defines how a participant is identified independently of its infrastructure, how trust is established at the message level rather than the connection level, how participants discover one another dynamically, and how interactions remain stable as systems evolve.

In the absence of such a layer, these concerns are implemented repeatedly in application code, infrastructure configuration, and organisational processes.

This fragmentation leads to inconsistency. Each system solves the same problem differently, and interoperability becomes difficult to achieve.

As we will see in **Chapter 2: Architecture**, ACP introduces this missing layer as part of a structured design that separates identity, discovery, trust, and transport concerns.

The Machine Collaboration Problem

To better understand the limitations of current approaches, it is useful to examine how distributed systems typically interact today.

In most modern architectures, systems communicate through **endpoint-based APIs**. A service exposes an HTTP endpoint, and another service invokes it. Authentication is handled through API keys, OAuth tokens, or infrastructure-level certificates. In asynchronous environments, messages are routed through brokers or queues, often using transport-specific conventions.

This model is effective when the number of integrations is small and when each interaction can be explicitly configured and maintained.

However, as the number of participants increases and as systems become more dynamic, this model begins to show its limitations.

Consider an environment in which hundreds or thousands of autonomous agents operate concurrently. Each agent may be deployed in a different infrastructure environment, may move between machines or clusters, may evolve independently, and may interact with multiple peers as part of complex workflows.

Under these conditions, several fundamental challenges emerge. These challenges are not specific to any particular tool or technology. They arise from the absence of a shared model for how autonomous systems collaborate.

Identity

The first challenge is identity.

Any system participating in a distributed interaction must be able to answer a basic question: **who is this participant?**

In many architectures, identity is inferred from endpoints. A URL, queue name, or network location becomes a proxy for identity. While this may be convenient, it introduces a structural weakness: endpoints describe **where** a system is, not **what** it is.

When infrastructure changes, and in modern environments, it changes frequently, endpoints change as well. If trust relationships are tied directly to endpoints, every infrastructure change becomes a security and configuration problem.

Autonomous systems require **persistent identities** that remain stable even when infrastructure evolves.

As we will explore in **Chapter 3: Identity Deep Dive**, ACP addresses this by separating identity from location and by anchoring trust in cryptographic identity documents rather than infrastructure assumptions.

Trust

The second challenge is trust.

Once a participant has been identified, other systems must be able to verify that messages genuinely originate from that participant.

Transport-level mechanisms such as TLS provide important guarantees about the communication channel. However, they do not always provide sufficient guarantees about the origin of application-level messages, particularly in environments where messages pass through intermediaries such as proxies, relays, or brokers.

In such scenarios, trust becomes fragmented. Different layers of the system provide partial guarantees, but there is no single, consistent model for end-to-end verification of authenticity.

Autonomous systems, therefore, require **cryptographic trust mechanisms embedded in the protocol itself**, so that messages can be verified independently of the path they take.

This is explored in detail in **Chapter 4: Message Model Deep Dive**, where message envelopes carry signatures and, where necessary, encrypted payloads that preserve trust across transports.

Discovery

The third challenge is discovery.

In small systems, endpoints can be configured manually. As systems scale, this approach becomes increasingly impractical.

Agents must be able to locate one another dynamically, without relying on manually maintained configuration. This becomes particularly important in environments where participants:

- are deployed dynamically
- operate across organisational boundaries
- move between infrastructure environments

Discovery mechanisms must therefore provide a reliable way to map stable identities to current endpoints, while remaining compatible with governance constraints and security requirements.

As we will examine in **Chapter 5: Discovery and the Well-Known Endpoint**, ACP introduces discovery as a first-class protocol concern, rather than leaving it to application-specific conventions.

Interoperability

The fourth challenge is interoperability.

Autonomous systems rarely operate within a single, uniform infrastructure. Some systems rely on HTTP APIs, others on message brokers, and others on event-driven architectures or specialised transports.

When interaction semantics are tied to a specific transport or framework, interoperability becomes difficult. Systems can only communicate effectively when they share the same underlying assumptions about how messages are structured and delivered.

A collaboration protocol must therefore remain independent of any particular transport, allowing the same interaction semantics to be preserved across different environments.

This requirement leads directly to ACP's **transport-independent message model**, discussed in **Chapter 6: Transport Binding Architecture**, where transport bindings are responsible only for delivery, not for defining semantics.

Governance

The final challenge is governance.

As systems interact across organisational boundaries, organisations must retain control over how their systems participate in those interactions.

This includes:

- defining which identities are trusted
- controlling how messages are routed
- enforcing policies on interaction patterns
- maintaining auditability and traceability

In traditional architectures, these concerns are often addressed through a combination of infrastructure controls and application-specific logic. However, this approach becomes increasingly difficult to manage as systems become more dynamic and interactions become more complex.

Without a consistent model for identity, trust, and discovery, governance mechanisms must be implemented repeatedly in different parts of the system, leading to inconsistency and increased operational risk.

As discussed earlier in this chapter, these concerns are particularly significant in regulated environments, where auditability and policy enforcement are critical.

ACP addresses this by making identity, trust, and discovery explicit protocol concepts, enabling governance to be applied consistently across interactions.

Taken together, these challenges reveal a gap in the current infrastructure for machine collaboration.

What is missing is not merely another messaging system or integration framework. What is missing is a **protocol layer designed specifically for autonomous participants**,a layer that treats identity, trust, discovery, interoperability, and governance as core concerns of interaction rather than incidental features of infrastructure.

The remainder of this book explores how ACP defines that layer, and how its design allows autonomous systems to collaborate reliably even as the environments in which they operate continue to evolve.

Governance and Risk in Autonomous Systems

The absence of a collaboration layer is not only an architectural issue. It is also a governance and risk management problem.

In enterprise environments, systems are subject to requirements such as identity assurance, access control, auditability, action traceability, and policy enforcement across boundaries.

When interactions are defined implicitly through endpoints and infrastructure, these requirements become difficult to enforce consistently.

If identity is tied to endpoints, infrastructure changes require revalidation of trust relationships. If message authenticity depends on transport, intermediary systems complicate verification. If discovery is manual, there is no reliable record of who can interact with whom and on what basis.

These challenges are not theoretical. They are well understood in fields such as financial services, healthcare, and critical infrastructure, where governance and audit requirements are strict.

Autonomous systems amplify these challenges. As the number of participants increases and interactions become more dynamic, the lack of a consistent collaboration model poses an operational risk.

ACP addresses this by making identity, trust, and discovery explicit protocol concepts rather than implicit infrastructure concerns.

Time as a First-Class Constraint

A defining characteristic of distributed systems is that they change over time.

Systems are deployed, updated, scaled, and retired. Infrastructure evolves. Organisational boundaries shift. New participants are introduced, and old ones disappear.

Most integration approaches treat time as an external factor. They assume that once a connection is established, it will remain valid.

In practice, this assumption rarely holds.

Designing for autonomous systems requires treating time as a first-class constraint. Systems must be able to maintain stable identities despite infrastructure changes, re-establish communication as endpoints evolve, verify trust independently of network topology, and adapt to new participants without manual reconfiguration.

These requirements cannot be satisfied solely at the transport or application logic levels. They require a protocol that explicitly models these concerns.

As we will explore in **Chapter 3: Identity Deep Dive**, persistent identity provides the foundation for stable trust relationships. In **Chapter 4: Message Model Deep Dive**, message-level semantics ensure that interactions remain meaningful across different transports. In **Chapter 5: Discovery and the Well-Known Endpoint**, dynamic discovery mechanisms allow systems to locate one another as they evolve.

Together, these elements form a model of interaction that remains stable over time.

Communication and Coordination

A final distinction is necessary.

Communication refers to the exchange of messages. Coordination refers to maintaining a shared understanding among participants.

Most distributed systems provide mechanisms for communication. Fewer provide explicit support for coordination.

Autonomous systems require coordination. They must manage workflows that span multiple participants, handle partial failures, and maintain a consistent state across interactions.

This requires more than the ability to send messages. It requires a protocol that defines how interactions are structured and verified.

From Tools to Protocols

Tools are designed to solve local problems. They improve developer productivity and simplify implementation.

Protocols define shared behaviour across systems. They enable interoperability between independently developed components.

ACP is designed as a protocol because the problem it addresses is not local. It is systemic. It emerges whenever autonomous systems must interact across boundaries.

What ACP Changes

ACP introduces a structured collaboration layer that makes identity, discovery, and trust explicit.

This does not eliminate complexity. Instead, it relocates complexity from implicit assumptions into explicit protocol constructs.

This shift makes systems easier to reason about, govern, and evolve.

The Trade-off

Introducing a protocol layer requires adopting new concepts. There is an initial cost in understanding identity documents, message envelopes, and discovery mechanisms.

However, this cost is offset by reduced long-term complexity.

Systems become less dependent on implicit assumptions. Interactions become more predictable. Changes can be managed without breaking existing relationships.

This trade-off reflects a shift in focus from short-term convenience to long-term stability.

A Mental Model

HTTP is for services.

ACP is for agents.

HTTP defines how clients and servers interact. ACP defines how autonomous participants collaborate.

What Follows

The chapters that follow develop these ideas in detail.

In **Chapter 1: Concepts**, we establish the vocabulary required to describe autonomous systems. In **Chapter 2: Architecture**, we examine how these concepts are organised into a layered model. Subsequent chapters explore identity, messaging, discovery, transport, and relay models.

Each builds on the foundation established here.

Closing

The purpose of this chapter is not to persuade, but to clarify.

If the challenges described here are familiar, then the need for a collaboration layer becomes apparent.

If they are not, then the rest of this book provides a framework for evaluating whether they will become relevant in your systems over time.

Getting Started with ACP

This guide introduces the fastest path to running your first ACP agent and sending your first message. It is intended to appear immediately after the **Preface** so readers can experience the protocol before diving into the deeper architectural chapters.

ACP is designed so that developers can begin experimenting with minimal setup and then gradually adopt more advanced capabilities as their systems evolve.

This section will guide you through:

1. Installing the ACP SDK
2. Installing the ACP CLI
3. Creating an agent identity
4. Running your first agent
5. Sending your first message

The example used throughout this guide is intentionally simple: two agents communicating directly using the `ping` capability.

System Requirements

ACP SDKs support multiple languages, but the easiest starting point is **Python**.

Recommended environment:

```
Python >= 3.14
pip >= 24
```

The examples assume a Unix-like shell environment, but the same commands work on Windows using PowerShell.

Optional tools for later chapters include:

- Docker (for relay deployments)
- Git (for cloning the repository)
- Node.js (for TypeScript SDK examples)
- Rust (for Rust SDK examples)

Installing the ACP Python SDK

The simplest way to begin is installing the Python SDK from PyPI.

```
pip install acp-sdk
```

This installs the ACP runtime that applications use to send and receive protocol messages.

The SDK provides:

- message envelope construction
- identity verification
- payload encryption
- discovery resolution
- transport communication

The runtime handles protocol mechanics automatically so developers can focus on application logic.

Installing the ACP CLI

The ACP command-line interface is distributed separately.

```
pip install acp-cli
```

The CLI allows developers and operators to interact directly with ACP systems.

Typical uses include:

- generating identities
- inspecting discovery metadata
- sending test messages
- debugging relay infrastructure

Later chapters explore the CLI in detail, but for now we will use it to create our first identity.

Creating Your First Agent Identity

Every ACP agent requires a cryptographic identity.

Create one using the CLI:

```
acp identity create
```

This command generates:

- an identity document
- signing keys
- encryption keys

The identity is stored in the default ACP configuration directory:

```
~/.acp/
```

The identity document represents the persistent identity of the agent. As explained in **Chapter 3**, this identity remains stable even if the infrastructure hosting the agent changes.

Creating Your First Agent

An ACP agent is simply an application hosting an ACP runtime.

Below is the smallest possible Python agent exposing a `ping` capability.

```
from acp import Agent

agent = Agent("agent:demo.ping")

@agent.route("ping")
def ping(payload):
    return {"status": "ok"}

agent.run()
```

Save this file as:

```
ping_agent.py
```

Then run the agent:

```
python ping_agent.py
```

The agent is now ready to receive ACP messages.

Sending Your First Message

Open another terminal and send a test message using the CLI.

```
acp message send agent:demo.ping ping
```

The CLI performs several steps automatically:

1. Constructs the ACP envelope
2. Signs the message with your identity key
3. Sends the message to the agent
4. Verifies the response signature

If everything is working correctly, you should see:

```
{"status": "ok"}
```

What Just Happened

Even though the example was simple, several important protocol operations occurred.

The SDK:

- constructed a message envelope
- signed the message using your identity
- delivered the message using the configured transport

The receiving agent:

- verified the signature
- invoked the capability handler
- signed the response

These behaviors are implemented automatically by the ACP runtime.

Later chapters explain the underlying mechanics in depth.

Next Steps

You have now completed the minimal ACP workflow.

The following chapters explore the protocol in much greater depth:

- **Chapter 1 - Concepts** explains the protocol's vocabulary.
- **Chapter 3 - Identity Deep Dive** explains how trust is established.
- **Chapter 6 - Transport Bindings** explains how messages travel across infrastructure.
- **Chapter 8 - Relay Architecture** introduces scalable routing networks.

The examples later in the book (such as the chess and poker agents) build on the same basic structure introduced here.

From this point forward, you already have everything required to begin experimenting with ACP.

Part I - Foundations

Chapter 1: Concepts

Chapter 2: Architecture

Chapter 3: Identity Deep Dive

Chapter 4: Message Model Deep Dive

Chapter 5: Discovery and the Well-Known Endpoint

This part establishes ACP's conceptual vocabulary and architectural core so later protocol details have a clear mental model.

Chapter 1 - Concepts

Before discussing architecture, transports, or SDK implementations, ACP requires a stable conceptual vocabulary. Protocols become easier to understand when readers can clearly identify the entities within the system and the responsibilities assigned to each. HTTP became widely accessible to developers because its core concepts,resources, requests, responses, and URLs,formed a simple and coherent mental model.

The Introduction argued that autonomous systems expose a missing layer in modern distributed computing: systems that appear to integrate successfully often become fragile as they evolve. The concepts introduced in this chapter can therefore be understood not merely as technical definitions, but as responses to those structural weaknesses.

ACP relies on a compact set of concepts that define how autonomous systems interact. These include **agent**, **identity**, **endpoint**, **message**, **capability**, **discovery**, **transport**, **relay**, and **overlay versus native deployment**.

These are not merely technical definitions. Each concept addresses a problem encountered when machines collaborate at scale.

The goal of this chapter is therefore not only to define these terms, but also to explain **why the protocol needs them** and how they fit together to form the collaboration model described throughout the rest of this book.

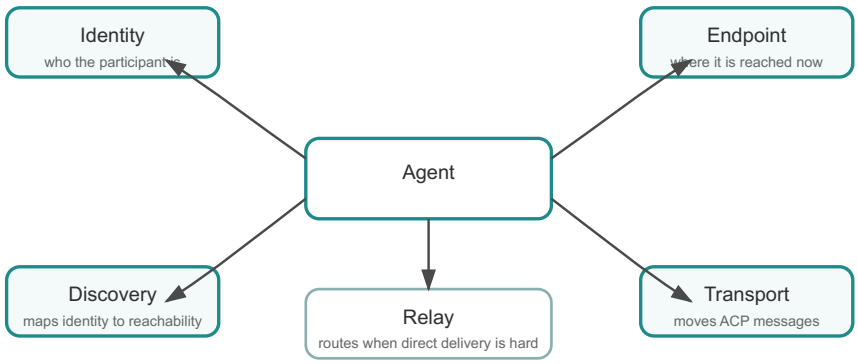


Figure 1.1. ACP concepts overview

1.1 Agents

The central participant in ACP is the **agent**.

An agent is an autonomous software participant capable of interacting with other systems through the protocol. Unlike a traditional service, which is often treated as a passive endpoint waiting for requests, an agent may actively initiate communication, coordinate tasks with peers, or participate in multi-step workflows.

Examples of agents may include:

- an AI research assistant coordinating experimental runs
- a trading algorithm negotiating transactions
- a software build system orchestrating distributed compilation
- a compliance service monitoring financial transactions
- a logistics agent coordinating shipments between suppliers

What distinguishes these systems is not the technology used to implement them but the role they play in the system. They operate as **participants in a collaborative network**, rather than isolated services performing single tasks.

As autonomous systems become more common, the number of such participants within a system grows dramatically. Instead of a handful of services communicating through fixed APIs, organisations may operate hundreds or thousands of agents interacting across different infrastructure environments.

ACP therefore assumes that the system is composed of **many autonomous participants** rather than a small number of static services.

1.2 Identity

If autonomous systems are going to collaborate safely, they must be able to answer a fundamental question:

Who is this participant?

As discussed in the Introduction, the absence of a stable identity model is one of the primary sources of fragility in distributed systems. In many distributed systems today, a service is effectively identified by where it lives. A URL, queue name, or topic path becomes both the route to the service and, informally, its identity. That seems convenient until the system moves. Containers are rescheduled, gateways are reconfigured, and services are relocated behind new infrastructure. If identity is tied directly to location, every infrastructure change becomes a trust and configuration problem.

ACP therefore introduces the concept of **persistent agent identity**.

An agent may be identified by a stable identifier, such as:

```
agent:ricardo.chess@demo
```

This identifier remains constant even when the underlying infrastructure changes. The identifier describes the participant, not the machine hosting it.

As we will see in **Chapter 3: Identity Deep Dive**, ACP reinforces this concept through **identity documents** that contain the cryptographic material necessary to verify an agent's messages.

By separating identity from infrastructure location, ACP allows trust relationships to remain stable even as systems evolve.

1.3 Endpoint

While identity describes **who an agent is**, the **endpoint** describes **where it can currently be reached**.

Endpoints are typically expressed as transport-specific addresses such as:

```
https://agent.example.com/acp
```

or

```
amqp://broker.example.com/agent.queue
```

Endpoints are inherently dynamic. They may change due to:

- infrastructure migration
- scaling events
- network reconfiguration
- security policy updates

ACP treats endpoints as **operational hints**, not as the definitive identity of the participant.

This separation between identity and endpoint allows agents to move across infrastructure environments without disrupting the trust relationships established with other participants.

Discovery mechanisms, discussed later in this chapter and explored further in **Chapter 5: Discovery and the Well-Known Endpoint**, provide the mapping between these two concepts.

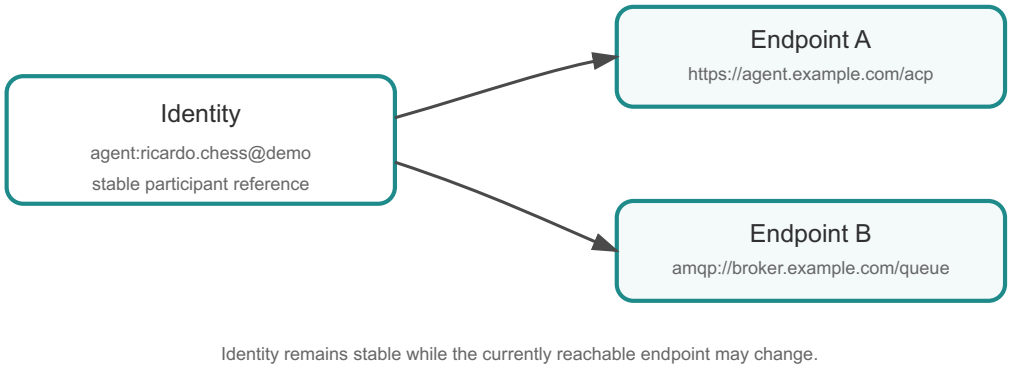


Figure 1.2. Identity versus endpoint

1.4 Messages

Agents collaborate by exchanging **messages**.

Unlike traditional application payloads, ACP messages are structured protocol artefacts that carry both semantic meaning and security guarantees. Each message contains an envelope that includes metadata describing the sender, the type of interaction being requested, and the cryptographic information required to verify authenticity.

A simplified representation of the message envelope looks like this:

```
ACP Envelope
 ├ message_id
 ├ agent_id
 ├ capability
 ├ signature
 └ encrypted payload
```

The envelope ensures that every message carries enough information to determine:

- who sent it
- what action it represents
- whether the message has been altered
- whether its contents should remain confidential

This design ensures the message's meaning and security remain intact even if it travels through different transport systems.

As we will explore in **Chapter 4: Message Model Deep Dive**, this envelope structure is the foundation of ACP's transport independence.

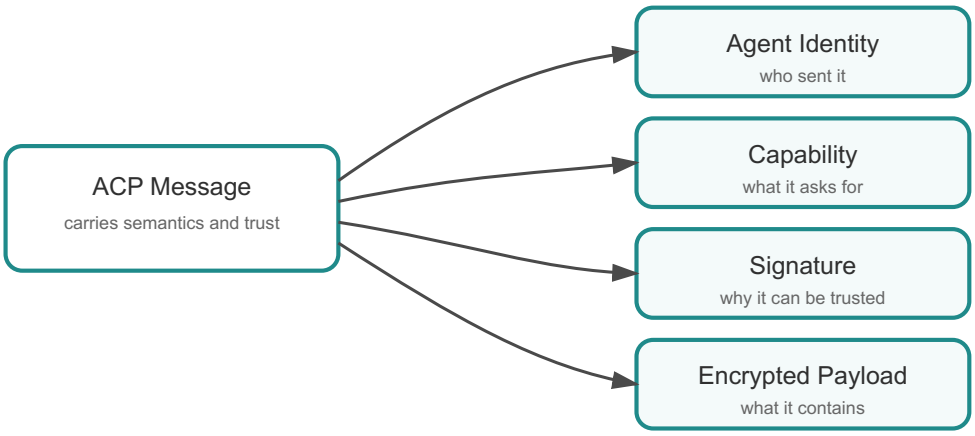


Figure 1.3. Message relationship model

1.5 Capabilities

In ACP, messages represent **capabilities**.

A capability describes the action or operation being requested. In traditional APIs, this information is often embedded in URL structures or application-specific conventions. ACP instead makes the semantic meaning explicit in the protocol.

Examples of capabilities might include:

- `ping` — test connectivity
- `execute_workflow` — initiate a distributed task
- `challenge` — request participation in a game
- `move` — submit a game action

Capabilities provide a common vocabulary for agent interaction. Because they are embedded directly in the protocol, they remain meaningful even when messages move across different transports.

Capabilities also support **evolution**. New capabilities can be introduced without breaking compatibility with existing systems, because agents can negotiate which capabilities they support.

This property is particularly important in environments where independent systems evolve at different speeds.

1.6 Discovery

Once agents have identities, the next challenge is determining how to reach them.

Discovery addresses a central challenge identified in the Introduction: how systems locate one another in environments where endpoints are not stable and cannot be maintained manually at scale. Autonomous systems operate in environments where participants may appear or disappear dynamically, infrastructure may change frequently, and systems may interact across organisational boundaries.

Without a discovery mechanism, every system would need a manually maintained map of every other system's endpoint. Such an approach becomes impractical as the number of participants grows.

ACP therefore introduces several discovery mechanisms that allow agents to locate one another dynamically.

One of the simplest of these mechanisms is the **well-known endpoint**:

```
GET /.well-known/acp
```

When queried, this endpoint returns metadata for contacting the agent. This allows clients to discover an agent with minimal prior configuration.

Discovery, therefore, acts as the bridge between **stable identity** and **dynamic infrastructure**.

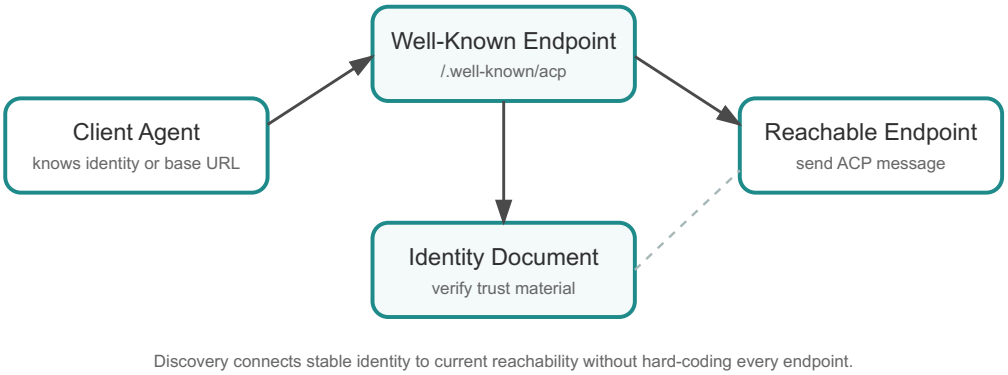


Figure 1.4. Discovery flow

1.7 Transport

Transport mechanisms move ACP messages between agents.

Examples include:

- HTTP(S) APIs
- AMQP broker messaging
- MQTT publish/subscribe systems
- relay networks

A central design goal of ACP is **transport independence**.

Many messaging systems couple their semantics tightly to a particular transport technology. ACP avoids this by embedding protocol semantics directly in the message envelope rather than relying on transport-specific conventions.

This means that the same ACP message can be delivered through multiple transport systems without changing its meaning.

As we will explore in **Chapter 6: Transport Binding Architecture**, transport bindings simply define how the envelope is carried across each delivery mechanism.

1.8 Relay

A **relay** is an intermediary that forwards ACP messages between agents.

Relays are useful when direct communication between agents is not possible. For example:

- agents may reside behind firewalls
- organisations may require controlled routing between networks
- systems may need to communicate across security domains

In such environments, the relay acts as a routing service rather than a semantic interpreter. It forwards messages according to protocol rules without needing to understand the payload.

Because ACP messages include signatures and optional encryption, relays can route them without access to their contents.

As we will see in **Chapter 8: Relay Architecture**, this property allows relay networks to span multiple organisations while preserving message security.

1.9 Overlay vs Native Operation

ACP can be deployed in two modes: **overlay mode** and **native mode**.

Overlay mode allows ACP to be layered on top of existing services. For example, an HTTP service may wrap its request handling logic with an ACP overlay adapter.

```
existing service
        +
ACP overlay wrapper
```

This approach allows organisations to adopt ACP gradually without redesigning their architecture.

Native mode represents the full protocol deployment model.

```
Agent
  |
ACP Runtime
  |
Discovery + Relay
  |
Transport
```

In native deployments, ACP becomes the primary communication fabric between agents.

The ability to move from overlay to native deployments is a key part of ACP's **low-friction adoption strategy**, discussed earlier in the Preface.

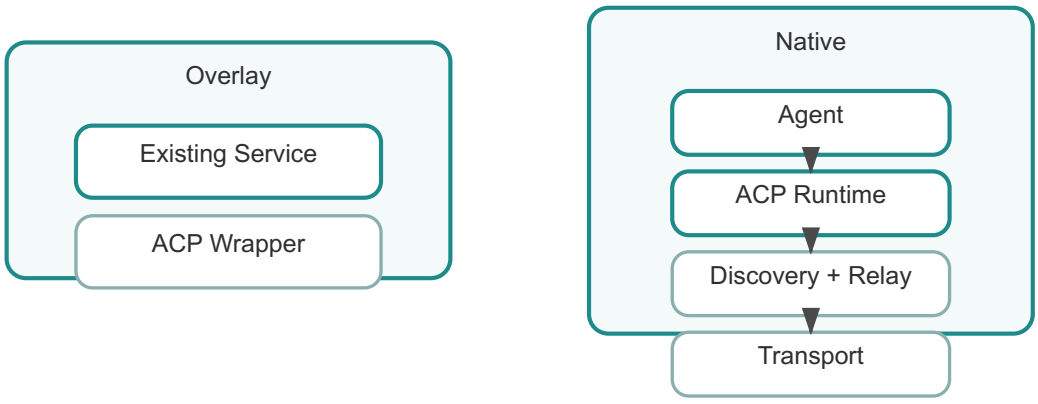


Figure 1.5. Overlay versus native

1.10 Summary

ACP relies on a small set of core concepts that define the protocol's vocabulary: agents, identities, endpoints, messages, capabilities, discovery, transports, relays, and deployment modes.

The most important conceptual distinction is the separation between **identity** and **endpoint**. Identity represents the participant, while the endpoint represents its current infrastructure location.

Messages, capabilities, and discovery transform these concepts into a working collaboration model. Messages carry meaning and trust, capabilities define the semantics of interaction, and discovery maps stable identities to reachable infrastructure.

These concepts collectively form the vocabulary required to address the coordination challenges outlined in the Introduction. With them established, the rest of the protocol architecture becomes easier to understand. The next chapter, **Chapter 2: Architecture**, explains how these ideas are organised into layers that allow ACP to operate across diverse environments while preserving consistent protocol semantics.

Chapter 2 - Architecture

The previous chapter introduced the conceptual vocabulary of ACP: agents, identities, messages, capabilities, discovery mechanisms, and the distinction between overlay and native deployments. These concepts describe **what exists in the system**. The next step is to understand **how these pieces are organised into a coherent architecture**.

The Introduction framed ACP as a response to the lack of a collaboration layer in autonomous systems. The purpose of this chapter is to make that claim concrete by showing how identity, trust, discovery, and transport are separated into layers that can evolve without collapsing into one another.

Architectural design becomes especially important in protocols intended to operate across many environments. ACP must function in small developer experiments, large enterprise deployments, and federated ecosystems spanning multiple organisations.

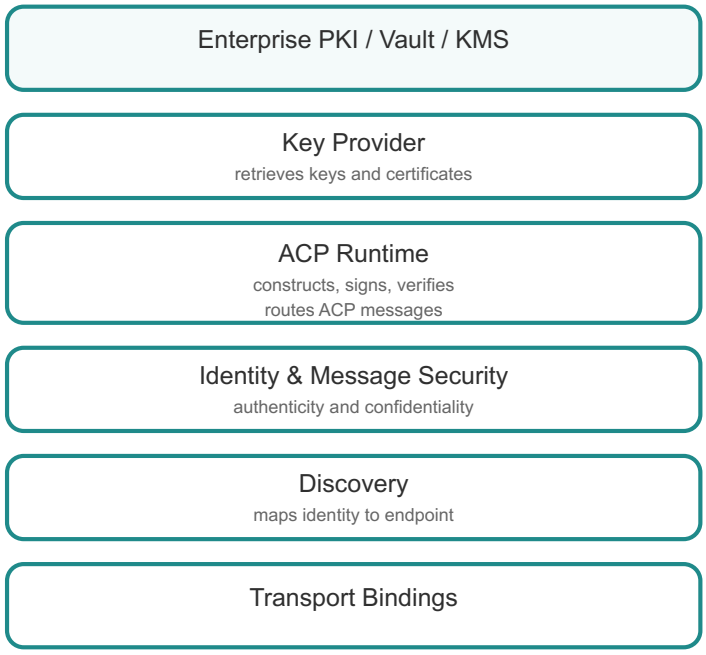


Figure 2.1. ACP layered architecture

2.1 Why Layered Architectures Matter

Layered architectures have a long history in distributed computing. The Internet itself relies on layered design: IP moves packets, TCP manages reliability, and HTTP defines application semantics.

ACP follows the same principle.

Different layers solve different classes of problems:

- governance of cryptographic material
- protocol runtime behaviour
- message security
- discovery and reachability
- message delivery across transports

By isolating these responsibilities, ACP allows infrastructure to evolve without breaking the protocol.

2.2 Enterprise Key Custody

Enterprises rarely allow applications to manage cryptographic keys directly. Instead, keys are managed by:

- PKI systems
- secret management platforms such as Vault
- cloud Key Management Services (KMS)
- Hardware Security Modules

ACP does not replace these systems. Instead, it integrates with them.

This decision dramatically reduces adoption friction because organisations can keep their existing security infrastructure while moving identity and trust into explicit protocol constructs.

2.3 Key Provider Abstraction

Between enterprise key custody and the runtime sits the **key provider abstraction**.

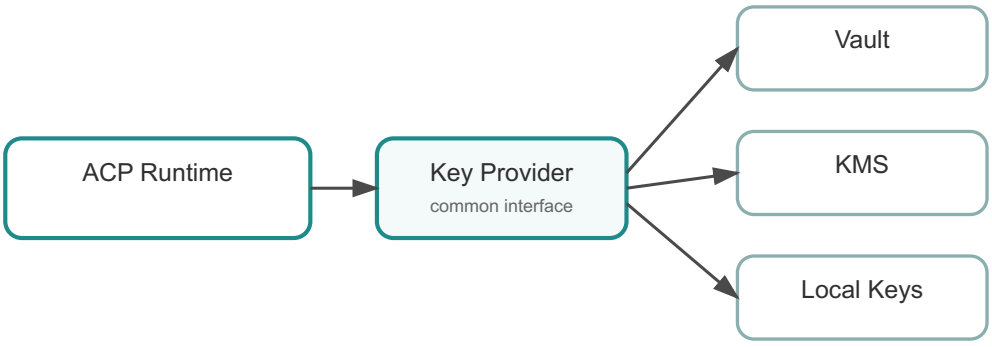


Figure 2.2. Key provider abstraction

This layer provides a consistent interface that allows the runtime to retrieve cryptographic material without knowing where that material is stored.

For example:

- development systems may use local key files
- production systems may use Vault or a KMS
- regulated environments may rely on HSM-backed PKI

The runtime simply asks the provider for the keys it needs.

2.4 The ACP Runtime

The runtime is the component that implements protocol behaviour in software.

It is responsible for:

- constructing ACP envelopes
- signing and verifying messages
- encrypting payloads when required
- interacting with discovery
- selecting transport bindings

SDK implementations for Python, Java, Rust, and TypeScript provide language-specific access to this runtime.

2.5 Identity and Message Security

ACP embeds identity and security directly into the message envelope.

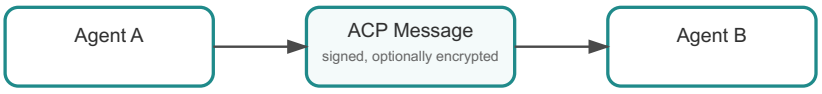


Figure 2.3. Message trust model

Because messages carry signatures and optional encryption, their authenticity and confidentiality do not depend solely on the transport channel.

This allows messages to pass through relays or intermediaries without compromising trust. It also aligns with the governance concerns identified in the Introduction, where auditability and verification must survive beyond a single network connection.

2.6 Discovery

Once agents have identities, they must determine where other agents can be reached.

Discovery mechanisms map stable identities to current endpoints.

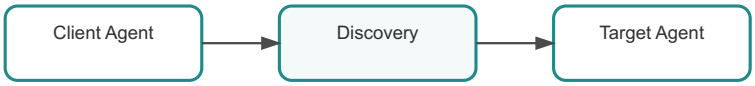


Figure 2.4. Discovery architecture

ACP supports several discovery approaches:

- static configuration
- discovery registries
- the well-known endpoint

Each method addresses different operational environments.

2.7 Transport Bindings

Finally, transport bindings move ACP messages across real infrastructure.

Common bindings include:

- HTTP(S)
- AMQP
- MQTT
- relay networks

Transport bindings define **how the envelope moves**, but they do not redefine protocol semantics.

2.8 Summary

ACP uses a layered architecture because the problems it solves,key custody, runtime behaviour, trust, discovery, and delivery,are distinct.

This separation allows ACP to operate across developer environments, enterprise infrastructures, and cross-organisation networks while maintaining a consistent protocol model.

The next chapter explores the identity model in detail and explains how ACP establishes trust between autonomous participants.

Chapter 3 - Identity Deep Dive

Identity is the core trust primitive of ACP. As discussed in **Chapter 1: Concepts** and **Chapter 2: Architecture**, ACP separates identity from endpoint so that trust does not collapse into infrastructure location. Autonomous systems cannot collaborate safely if they cannot answer a basic question: **who is sending this message?**

The Introduction argued that many distributed systems answer this question only indirectly, through assumptions about their infrastructure. ACP takes a different approach. It treats identity as a **first-class protocol concept** rather than an infrastructure side effect.

In modern distributed systems, that question is often answered indirectly through infrastructure assumptions. A request arrives over HTTPS from a known host, or a message appears on a trusted queue, and the infrastructure context is treated as sufficient proof of identity. This approach works reasonably well when systems are static and centrally managed. However, autonomous systems behave differently. They move across infrastructure, scale dynamically, and frequently interact across organisational boundaries.

Trust relationships that depend entirely on infrastructure location quickly become fragile when services move, networks change, or messages pass through intermediaries.

ACP therefore takes a different approach: identity is treated as a **first-class protocol concept** rather than an infrastructure side effect.

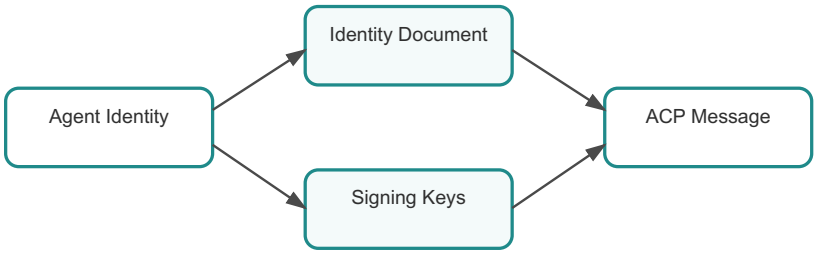


Figure 3.1. Identity architecture

3.1 Identity as a Protocol Primitive

In ACP, every participant has a **persistent identity**. An identity is not tied to a specific host, container, or network address. Instead, it represents the participant itself.

Example:

```
agent:ricardo.chess@demo
```

This identifier remains stable even when:

- the agent moves to a different machine
- the service is redeployed
- the communication path changes
- the infrastructure hosting the agent evolves

This design reflects an important observation from distributed systems research: **location is not identity**.

Many large-scale architectures eventually rediscover this principle. Service meshes introduce service identities that are separate from IP addresses. Zero-trust security models treat identities rather than network locations as the basis for trust decisions. Modern distributed identity frameworks similarly emphasise stable identities over infrastructure identifiers.

ACP adopts the same principle at the protocol level.

3.2 Identity Documents

A stable identifier alone is not enough to establish trust. Other participants need a way to verify that a message claiming to come from an agent actually originates from that agent.

ACP therefore introduces the **identity document**.

An identity document is a signed artefact containing the agent's public cryptographic material. At a minimum it includes:

- the agent identifier
- the public signing key
- the public encryption key
- optional metadata describing the agent

A simplified identity document might look like this:

```
{
  "agent_id": "agent:ricardo.chess@demo",
  "signing_key": "public-key-data",
  "encryption_key": "public-key-data"
}
```

This document allows other agents to verify message signatures and establish trust relationships.

The identity document, therefore, performs a role similar to a certificate in PKI systems. It connects a stable identifier with verifiable cryptographic material.

3.3 Identity vs Endpoint Trust

One of the most important distinctions in ACP is the difference between **agent identity** and **endpoint trust**.

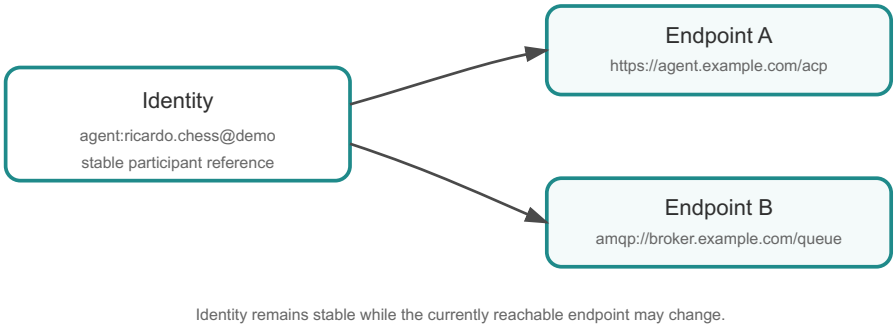


Figure 3.2. Identity vs endpoint

Agent identity answers the question:

Who is the participant that produced this message?

Endpoint trust answers a different question:

Is the network endpoint currently delivering the message trustworthy?

Transport mechanisms such as TLS or mTLS address endpoint trust. They ensure that the communication channel is protected and that the endpoint presenting the certificate is authentic.

ACP identity operates at a different layer. It ensures that the message itself originates from the correct participant, regardless of the path it took through the infrastructure.

This layered trust model provides several advantages.

First, it allows ACP to integrate with enterprise security infrastructure rather than replacing it. Organisations can continue to rely on PKI, certificate authorities, and secret management systems.

Second, it allows trust to survive infrastructure changes. An agent may move to a different endpoint without losing its identity.

Third, it allows messages to pass through intermediaries without losing their trust properties.

3.4 Identity and Message Integrity

Identity becomes operational through **message signatures**.

Every ACP message may be signed using the sender's private key. The recipient verifies the signature using the public key contained in the sender's identity document.

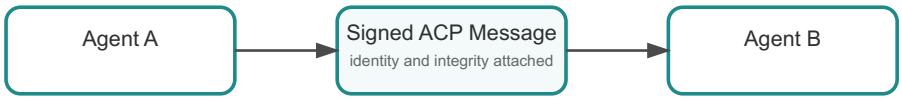


Figure 3.3. Message signature model

This mechanism ensures two important properties:

- **authenticity** — the message originated from the claimed agent
- **integrity** — the message contents have not been altered

These guarantees remain valid even if the message travels through intermediaries such as relays or brokers.

This property becomes particularly important in multi-hop architectures. In such systems, the transport channel may change several times before the message reaches its destination. Transport security alone cannot guarantee end-to-end authenticity. Message-level signatures solve that problem.

3.5 Identity and Mobility

One of the most powerful consequences of ACP's identity model is support for **agent mobility**.

Modern infrastructure environments are highly dynamic. Container orchestration platforms reschedule workloads automatically. Cloud systems scale services horizontally. Network routing paths change frequently.

If identity were tied directly to endpoint location, every infrastructure change would require re-establishing trust relationships.

ACP avoids this problem by anchoring trust in identity rather than infrastructure.

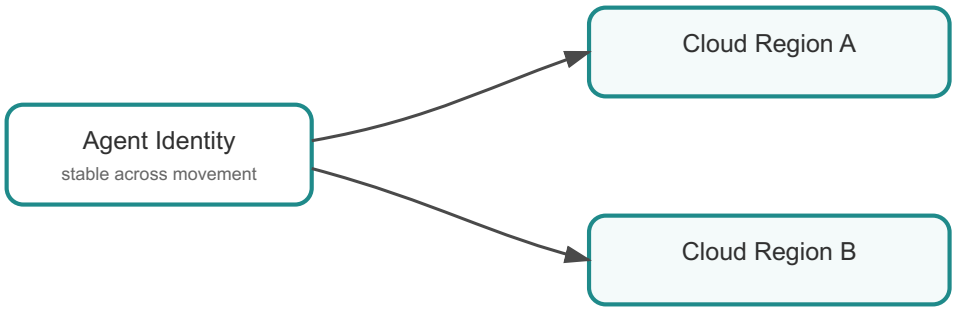


Figure 3.4. Agent mobility

When an agent moves, discovery mechanisms update the mapping between identity and endpoint. Other agents continue interacting with the same identity without needing to reconfigure trust relationships.

This property greatly simplifies operational management in large distributed systems.

3.6 Identity and Governance

Persistent identity also enables governance.

Organisations can define policies such as:

- which identities are trusted
- which capabilities particular identities may invoke
- which discovery registries or relays identities may use

Without persistent identity, these policies would need to be expressed purely in terms of infrastructure rules such as IP addresses or endpoint URLs. Those rules become difficult to maintain as systems evolve.

Identity allows policy to be expressed at the level of **participants** rather than infrastructure components.

This distinction is especially important in cross-organisation environments where infrastructure details are often hidden behind network boundaries.

3.7 Identity in the Wider Ecosystem

The importance of stable identity has been recognised across many areas of distributed computing.

Zero-trust architectures emphasise identity-based security rather than network location. Service mesh technologies assign identities to workloads independent of IP addresses. Modern authentication frameworks similarly focus on identity as the core trust primitive.

ACP extends this principle into the domain of machine collaboration protocols.

By embedding identity directly into the protocol semantics, ACP ensures that trust relationships remain meaningful even as infrastructure, transports, and routing paths evolve.

3.8 Summary

ACP treats identity as the core trust primitive of autonomous collaboration.

Instead of identifying participants by their infrastructure location, ACP assigns each agent a persistent identity that remains stable across deployments and infrastructure changes.

Identity documents link these identifiers to verifiable cryptographic material, enabling messages to be authenticated and verified.

The distinction between agent identity and endpoint trust allows ACP to integrate with enterprise security infrastructure while preserving protocol-level trust semantics.

By anchoring trust in identity rather than infrastructure, ACP enables mobility, governance, and secure collaboration across dynamic distributed systems.

Chapter 4 - Message Model Deep Dive

The ACP message model is where the protocol's trust and collaboration semantics become concrete. As discussed in **Chapter 3: Identity Deep Dive**, identity only becomes operational when it is bound to verifiable protocol artefacts.

In the Introduction, one of the central arguments was that communication alone is not sufficient for autonomous systems. Messages must preserve identity, meaning, and trust as interactions move across changing infrastructure. The ACP message model is where that requirement becomes operational.

In traditional distributed systems, application payloads are often treated as opaque blobs transported across network channels, such as HTTP requests, message queues, or event streams. While this approach works for simple integrations, it becomes fragile in complex multi-agent ecosystems.

Autonomous systems require something stronger.

They require messages that carry **identity, meaning, and trust** together as a single protocol artefact.

ACP, therefore, treats the **message envelope** as the fundamental unit of collaboration.

Rather than embedding semantics in transport conventions or application-specific payload formats, ACP places the meaning of an interaction directly in the message structure itself.

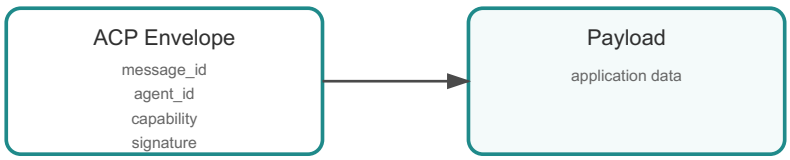


Figure 4.1. ACP message envelope

This design addresses several systemic problems in distributed collaboration systems:

- establishing trust between participants
- preserving semantics across different transports
- supporting asynchronous and distributed workflows
- enabling reliable tracing and observability

These requirements become increasingly important as the number of autonomous participants grows.

4.1 Why Messages Matter in Autonomous Systems

In classical client-server systems, communication typically follows a relatively simple model. A client issues a request to a known endpoint and receives a response. Identity is often inferred from infrastructure context such as TLS certificates, API tokens, or network boundaries.

Autonomous systems introduce a more complex environment.

Participants may:

- communicate asynchronously
- operate across different infrastructures
- interact through relays or broker systems
- negotiate capabilities dynamically

In such environments, the transport channel alone cannot fully describe the meaning of an interaction.

Researchers studying distributed system reliability have repeatedly observed that tightly coupling semantics to transport conventions leads to brittle systems. Martin Fowler, in his discussions of enterprise messaging patterns, notes that loosely structured message payloads often lead to fragile integration layers that become difficult to evolve.

ACP addresses this problem by introducing a **structured message envelope** that travels with the payload regardless of transport.

4.2 Structure of the ACP Message Envelope

An ACP message consists of two main components:

1. the **envelope**, which contains protocol metadata
2. the **payload**, which contains application data

A simplified representation looks like this:

```
ACP Envelope
 ├ message_id
 ├ agent_id
 ├ capability
 ├ timestamp
 ├ signature
 └ encrypted payload
```

Each field exists because the protocol must solve a specific problem.

4.3 Message Identifier

Distributed systems frequently experience retries, duplicate deliveries, and asynchronous workflows. Without a stable identifier, a receiving agent may have difficulty distinguishing between:

- a repeated delivery of the same message
- a new message with similar content

The **message_id** field solves this problem.

Each ACP message carries a globally unique identifier. This identifier enables:

- deduplication of repeated messages
- correlation between request and response messages
- tracing of interactions across distributed systems

For operations teams, message identifiers provide a powerful observability tool. Monitoring systems can reconstruct message flows across agents, relays, and transport systems.

4.4 Agent Identifier

The **agent_id** field identifies the sender of the message.

As explained in Chapter 3, this identifier corresponds to the agent's persistent identity rather than the infrastructure location from which the message was sent.

This distinction is critical.

A message may pass through several layers of infrastructure before reaching its destination:

```
Agent A → Relay → Broker → Relay → Agent B
```

Without a protocol-level identifier, the recipient might only know which infrastructure node forwarded the message rather than the participant that originally created it.

By embedding the agent identity directly in the message envelope, ACP ensures that the sender's identity remains visible regardless of the transport path.

4.5 Capability

The **capability** field defines the semantic intent of the message.

Many distributed systems implicitly encode semantic meaning in transport conventions such as:

- HTTP endpoints (`/execute`, `/update`)

- broker topic names
- application-specific payload formats

While convenient initially, these approaches make interoperability difficult because the meaning of the interaction becomes tied to the specific transport technology.

ACP instead introduces an explicit capability field.

Examples of capabilities might include:

- `ping`
- `execute_workflow`
- `challenge`
- `move`
- `request_quote`

Because capabilities are embedded in the protocol, they remain meaningful even when the transport changes.

Capabilities also support **protocol evolution**. New capabilities can be introduced without breaking compatibility with older systems.

4.6 Timestamp

Distributed systems operate in environments where time matters.

Messages may be delayed, replayed, or processed asynchronously. Without a temporal context, it becomes difficult to detect anomalies such as replay attacks or stale messages.

The **timestamp** field provides that context.

While timestamps alone do not solve every temporal problem, they provide important signals for:

- replay detection
- operational debugging
- audit trails

Many enterprise governance systems rely on such metadata when reconstructing system activity.

4.7 Signature

The **signature** field provides authenticity and integrity.

When an agent sends a message, the runtime signs the envelope using the agent's private key. The recipient verifies the signature using the public key contained in the sender's identity document.

This process confirms two things:

1. the message originated from the claimed agent
2. the message has not been altered in transit

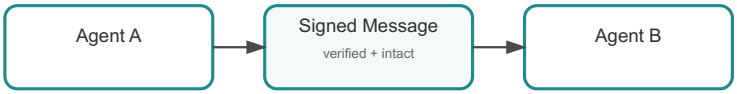


Figure 4.2. Message trust model

This property is especially important in architectures where messages pass through intermediaries such as relays or broker systems.

Transport security alone cannot guarantee end-to-end authenticity when messages traverse multiple systems. Protocol-level signatures provide that guarantee.

4.8 Payload Encryption

In some environments, confidentiality must extend beyond the transport layer.

For example, a relay may route messages between organisations without being authorised to inspect their contents.

ACP, therefore, allows payload encryption using the recipient's public key.

Only the intended recipient can decrypt the payload.

This capability allows ACP messages to travel across infrastructure that may not be fully trusted while preserving confidentiality.

4.9 Self-Contained Semantics

Taken together, the fields in the ACP message envelope create a **self-contained protocol artefact**.

The message carries:

- the identity of the sender
- the semantics of the interaction
- the cryptographic proof of authenticity

- the payload data
- the temporal context of the interaction

This design ensures the message remains meaningful regardless of the delivery mechanism.

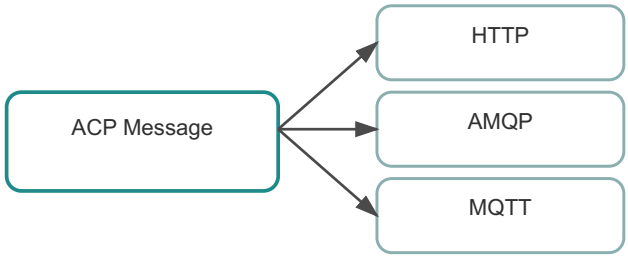


Figure 4.3. Envelope portability

A message sent via HTTP can be forwarded through a relay or broker without losing its semantics.

This property is the foundation of ACP's **transport independence**.

4.10 Message Evolution and Compatibility

Another important property of the ACP message model is its support for **evolution**.

Distributed ecosystems rarely upgrade all participants simultaneously. Systems evolve at different rates, and new capabilities must be introduced gradually.

Because ACP messages include explicit capability identifiers and structured metadata, new features can be introduced without forcing immediate upgrades across the entire ecosystem.

Agents that do not recognise a capability can safely ignore it or negotiate alternatives.

This approach aligns with lessons learned from Internet protocols such as HTTP and SMTP, which achieved long-term success partly because they were designed to evolve gradually.

4.11 Summary

The ACP message envelope is the protocol's operational core. It binds together identity, semantic meaning, and cryptographic trust in a transport-independent structure.

Each field addresses a specific challenge faced by autonomous systems. Message identifiers support reliability and tracing. Agent identifiers anchor trust in persistent

identities. Capabilities define the semantic intent of interactions. Signatures ensure authenticity and integrity, while optional encryption protects confidentiality.

By embedding these properties directly into the protocol, ACP ensures that messages remain meaningful across diverse infrastructures.

In the next chapter we examine how discovery mechanisms use these trust-bearing messages to locate agents dynamically across distributed environments.

Chapter 5 - Discovery and the Well-Known Endpoint

Stable identities and secure messages are necessary for machine collaboration, but they are not sufficient on their own. As we explored in **Chapter 3: Identity Deep Dive**, ACP establishes who a participant is through persistent identity. In **Chapter 4: Message Model Deep Dive**, we saw how the message envelope binds that identity to a verifiable and semantically meaningful interaction. Yet a fundamental question remains:

Where should a message actually be sent?

This question may appear trivial in small systems where endpoints are static and well-known. However, in the environments where ACP is intended to operate,large distributed ecosystems of autonomous agents,the answer is rarely obvious. Agents may appear, disappear, relocate, or change infrastructure dynamically. Networks may be segmented by organisational boundaries, and routing paths may change as systems scale or evolve.

The Introduction identified discovery as one of the protocol concerns that existing integration models leave to manual configuration or local convention. This chapter examines how ACP turns that concern into an explicit layer.

The mechanism that solves this problem is **discovery**.

Discovery connects **identity** with **reachability**. It allows agents to determine where another participant can currently be contacted without requiring every participant to maintain a constantly updated map of endpoints.

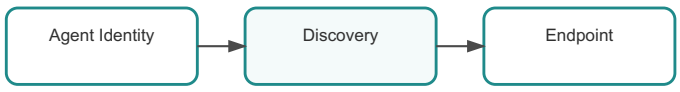


Figure 5.1. Discovery concept

In practice, discovery turns ACP from a theoretical trust model into a working communication system.

5.1 The Discovery Problem in Distributed Systems

The discovery problem has long been recognised in distributed computing. Early service-oriented architectures relied heavily on configuration files and manually maintained endpoint registries. As systems grew larger and more dynamic, these approaches became increasingly fragile.

Modern cloud environments illustrate why.

A single logical service may run across multiple containers, virtual machines, or clusters. Infrastructure orchestration platforms constantly reschedule workloads. Network topologies evolve as organisations introduce gateways, service meshes, or security zones.

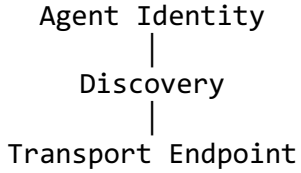
If every participant in such a system had to maintain direct knowledge of every other participant's current location, the operational burden would quickly become unmanageable.

Researchers in distributed systems have frequently observed that **service discovery** is one of the most fundamental infrastructure capabilities required for large-scale systems. Technologies such as DNS, service registries, and service meshes all emerged in response to the same underlying challenge: mapping stable identities to changing infrastructure.

ACP approaches the discovery problem with the same philosophy used throughout the protocol: separate the **stable identity of the participant** from the **dynamic infrastructure that hosts it**.

5.2 Discovery as a Protocol Layer

In the layered architecture introduced in **Chapter 2**, discovery sits between the identity and transport layers. Its role is to translate a stable agent identifier into the information required to deliver a message.



This separation is intentional.

Identity establishes trust. Transport moves messages. Discovery connects the two.

Without discovery, agents would need to be configured directly for every peer. Without identity, discovery results could not be trusted. Without transport bindings, discovered endpoints could not be used to deliver messages.

By isolating discovery as its own layer, ACP allows these responsibilities to evolve independently.

5.3 Discovery Models

Different environments require different discovery strategies. ACP therefore supports multiple discovery models rather than enforcing a single mechanism.

5.3.1 Static Configuration

The simplest approach is static configuration. In this model, an agent's identity is mapped to a known endpoint through configuration files or deployment metadata.

Static configuration is useful for:

- development environments
- small systems with few participants
- tightly controlled internal networks

However, static configuration becomes difficult to maintain when systems evolve rapidly.

5.3.2 Registry-Based Discovery

Larger deployments often rely on **discovery registries**.

A discovery registry maintains mappings between agent identities and communication hints. Agents register their endpoints with the registry, and other agents query it to locate peers.

Registry-based discovery provides several advantages:

- centralised visibility of participating agents
- governance and policy enforcement
- auditability for enterprise environments

However, registries introduce operational complexity and infrastructure overhead.

5.3.3 Well-Known Endpoint Discovery

ACP introduces a lightweight alternative: the **well-known endpoint**.

The well-known endpoint provides a simple bootstrap discovery mechanism using a standardised HTTP path.

```
GET /.well-known/acp
```

When queried, this endpoint returns metadata describing the agent's identity and communication hints.

Example:

```
{
  "agent_id": "agent:demo.service",
  "identity_document": "https://agent.example.com/identity.json",
  "transports": {
    "http": {
      "endpoint": "https://agent.example.com/acp"
    }
```

```
  },
  "version": "1.0"
}
```

This design is inspired by the broader "well-known URI" convention used in many Internet protocols.

5.4 Why the Well-Known Endpoint Matters

The well-known endpoint serves two important purposes.

First, it enables **bootstrap discovery**. An agent that knows only a base URL can retrieve enough information to initiate secure communication.

Second, it supports **low-friction adoption**.

As explained in the Preface, one of ACP's central design goals is pragmatic adoption. Organisations must be able to experiment with the protocol without deploying complex infrastructure from the start. The well-known endpoint allows an existing service to become ACP-discoverable simply by publishing a small metadata document.

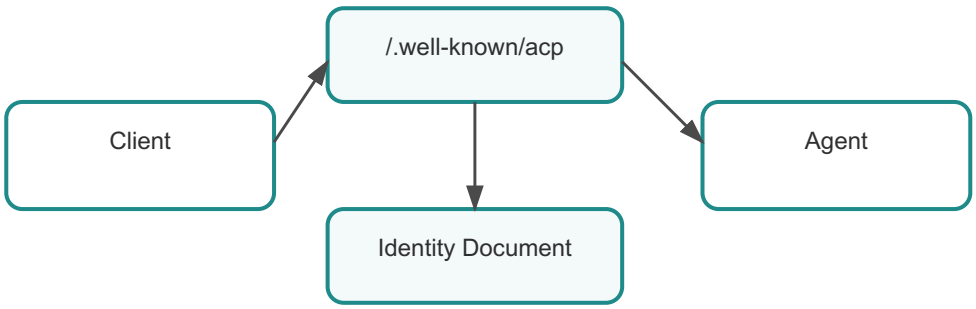


Figure 5.2. Well-known endpoint discovery

In other words, the well-known endpoint reduces the barrier to entry while remaining compatible with more advanced discovery mechanisms.

5.5 Authority and Trust in Discovery

Discovery mechanisms must also address a subtle question: **which discovery information should be trusted?**

Different sources of discovery information may exist simultaneously:

- local configuration
- enterprise discovery registries

- well-known endpoint metadata
- cached discovery data

ACP therefore benefits from a clear **authority model**.

Typically:

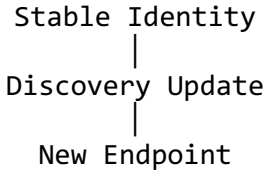
1. Local configuration has the highest authority.
2. Enterprise registries provide governed mappings.
3. Well-known metadata provides bootstrap hints.
4. Cached information has the lowest authority.

This hierarchy ensures that discovery results can be overridden when governance policies require it.

5.6 Discovery and Identity Mobility

One of the most important consequences of ACP discovery is support for **identity mobility**.

Agents may move between infrastructure environments without changing their identity. When this occurs, discovery information is updated to reflect the new endpoint.



Other agents continue interacting with the same identity while automatically resolving the new location.

This capability is particularly important in modern cloud environments where infrastructure mobility is common.

5.7 Discovery in Multi-Organisation Ecosystems

Discovery becomes even more important in cross-organisation ecosystems.

Organisations may operate their own registries or relay networks while still participating in a shared collaboration environment. Discovery mechanisms allow these systems to locate one another without requiring centralised control over all participants.

This property enables federated agent ecosystems where organisations maintain autonomy while still cooperating through a common protocol.

5.8 Summary

Discovery connects ACP's trust model to operational reality.

While identity establishes who a participant is and message envelopes preserve the meaning and integrity of interactions, discovery determines **where those interactions should be delivered**.

ACP supports multiple discovery mechanisms, including static configuration, registries, and well-known endpoints, because different environments require different balances of flexibility and governance.

The well-known endpoint is particularly important because it provides a low-friction bootstrap mechanism that aligns with ACP's incremental adoption strategy.

By allowing stable identities to map dynamically to changing infrastructure, discovery enables ACP to support mobile agents, federated ecosystems, and large-scale autonomous collaboration.

Part II - Protocol and Runtime

Chapter 6: Transport Binding Architecture

Chapter 7: Security Architecture

Chapter 8: Relay Architecture

Chapter 9: Capability Negotiation

Chapter 10: SDK Architecture

This part explains the protocol machinery itself: transport bindings, security, relays, negotiation, and runtime responsibilities.

Chapter 6 - Transport Binding Architecture

In the previous chapters we examined the conceptual foundations of ACP: the identity model, the message envelope, and the discovery mechanisms that allow autonomous participants to locate each other dynamically. These chapters established the *semantic* layer, the rules that define how autonomous systems collaborate and how trust is established between them.

However, once two agents know **who** they are communicating with and **what** interaction they want to perform, a practical question remains:

How does the message actually travel between them?

The answer lies in **transport bindings**.

At this point, the discussion moves from conceptual design into implementation. As outlined in the Introduction, interaction between autonomous systems must remain stable even as infrastructure evolves. Transport is therefore not just a delivery concern, but the point at which implementation choices can either preserve or undermine that stability.

Transport bindings are the architectural mechanism that allows ACP messages to move across different communication infrastructures without changing the protocol's meaning. This design choice reflects an important lesson from decades of distributed system evolution: communication infrastructure changes much faster than the semantics of collaboration.

As noted in the Preface, modern systems operate across a heterogeneous landscape of technologies, HTTP APIs, message brokers, event streams, and relay networks. If a protocol tightly couples its semantics to any one of these infrastructures, it becomes brittle and difficult to evolve.

ACP therefore takes a different approach: **separate the meaning of the message from the mechanism used to deliver it.**

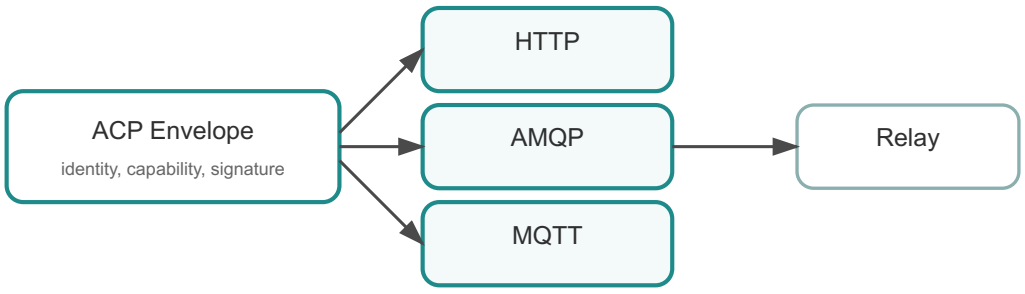


Figure 6.1. ACP transport architecture

6.1 The Historical Problem with Transport-Coupled Protocols

Many distributed systems inadvertently embed protocol semantics inside transport mechanisms. For example, HTTP-based APIs often encode meaning through a combination of:

- URL paths
- HTTP verbs
- JSON payload conventions
- endpoint naming schemes

While this approach is convenient for small systems, it creates challenges when infrastructure changes. If the system later needs to integrate with a broker-based architecture or an event-driven environment, those transport-specific assumptions become obstacles.

This problem has been widely recognised in enterprise architecture. Patterns described in works such as *Enterprise Integration Patterns* by Hohpe and Woolf highlight the difficulties of tightly coupling message semantics to transport infrastructure. Messaging middleware often introduces envelopes and metadata layers precisely to decouple meaning from delivery.

ACP extends this idea further by making the **message envelope the primary protocol artefact**.

Transport becomes a delivery channel, not a semantic authority.

6.2 The Role of Transport Bindings

A **transport binding** defines how an ACP message envelope is carried over a specific communication system. The envelope itself remains unchanged; the binding simply defines how it is serialised, transmitted, and received.

In practical terms, a transport binding answers questions such as:

- How is the envelope encoded for this transport?
- How is the destination address represented?
- How are responses routed back to the sender?

Because ACP messages already contain identity, capability, and signature information, the transport does not need to interpret the message. It simply delivers it.

This design ensures that ACP semantics remain stable even when the infrastructure evolves.

6.3 Example Transport Bindings

ACP currently supports several transport environments.

6.3.1 HTTP Binding

In an HTTP binding, the ACP envelope is transmitted as the request body. The endpoint may correspond to the address discovered through the well-known endpoint described in Chapter 5.

```
Agent A → HTTPS Request → Agent B
```

The message envelope is serialised as JSON and delivered through the HTTP request body.

6.3.2 AMQP Binding

In a broker-based architecture, the same ACP envelope may be placed inside an AMQP message and delivered through a queue or exchange.

```
Agent A → Broker → Agent B
```

The broker routes the message according to its own topology, but the ACP semantics remain unchanged.

6.3.3 MQTT Binding

Lightweight publish/subscribe infrastructures such as MQTT can also carry ACP messages. The envelope is published to a topic associated with the destination agent.

```
Agent A → MQTT Topic → Agent B
```

6.3.4 Relay Binding

In some environments, direct communication is not possible. ACP relays can forward envelopes between agents while preserving protocol semantics.

```
Agent A → Relay → Agent B
```

Relays provide routing functionality without needing to understand the message payload.

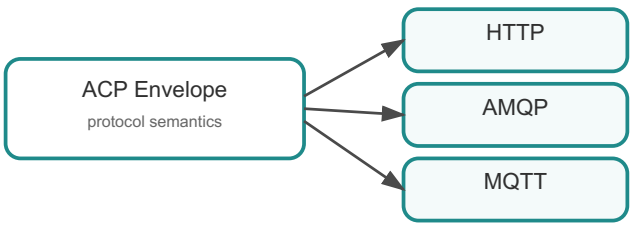


Figure 6.2. Transport binding mapping

6.4 Why Transport Independence Matters

Transport independence provides several strategic advantages.

First, it allows ACP to function in heterogeneous environments. Different organisations may operate different communication infrastructures yet still collaborate through a shared protocol.

Second, it allows systems to evolve gradually. A service that initially communicates through HTTP may later transition to a broker architecture without rewriting the protocol logic.

Third, it improves resilience. Systems can route messages through alternative transport paths if a particular infrastructure component fails.

These properties align closely with the realities described in the Preface: modern distributed systems are dynamic and long-lived. The infrastructure supporting them will inevitably change.

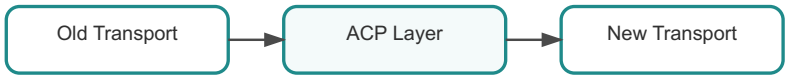


Figure 6.3. Transport evolution

6.5 Multi-Transport Ecosystems

In large deployments, it is common for multiple transport mechanisms to coexist.

An ACP ecosystem may include:

- HTTP-based services
- broker-based workflows
- event-driven pipelines
- relay networks bridging security domains

ACP messages can flow across these environments while preserving their semantics.

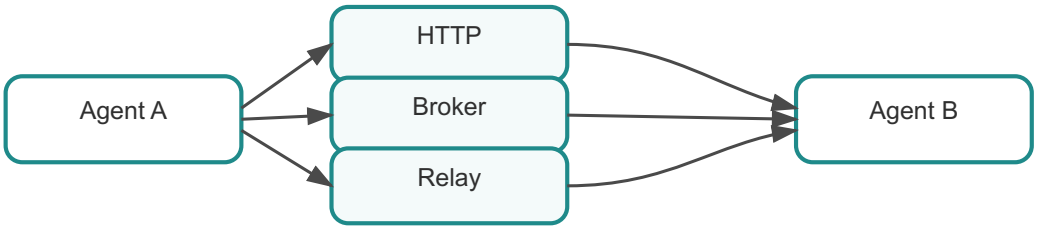


Figure 6.4. Multi transport ecosystem

This ability to operate across multiple infrastructures is one of the reasons ACP can scale beyond a single technology stack.

6.6 Operational Considerations

Although transport bindings preserve protocol semantics, each transport introduces operational considerations.

HTTP transports often rely on request/response patterns and may require gateway infrastructure. Broker-based systems provide asynchronous delivery but require queue management and routing configuration. Event-stream transports emphasise throughput and scalability.

ACP deliberately avoids enforcing a single operational model. Instead, it provides a protocol layer that can adapt to these environments while preserving the core semantics of autonomous collaboration.

This flexibility allows organisations to choose infrastructure according to operational needs rather than protocol limitations.

6.7 Summary

Transport bindings allow ACP messages to travel across real communication infrastructure without redefining the protocol's semantics.

By separating protocol semantics from delivery mechanisms, ACP avoids becoming tightly coupled to any single transport technology. The same message envelope can be delivered through HTTP APIs, message brokers, event streams, or relay networks.

This separation makes ACP resilient to infrastructure change and enables collaboration across heterogeneous systems.

In the next chapter, we explore the **security architecture** that protects these interactions, including how transport security and protocol-level cryptographic signatures work together to establish trust between autonomous participants.

Chapter 7 - Security Architecture

Security in ACP is intentionally layered. As introduced in **Chapter 2: Architecture**, layered design allows different security concerns to evolve independently. This architectural choice reflects a broader reality highlighted in the Preface: autonomous systems operate in environments that are dynamic, distributed, and often span multiple organisational boundaries.

In such environments, relying on a single security mechanism is rarely sufficient. Messages may travel through proxies, relays, brokers, and other intermediaries. Infrastructure layers may be owned by different teams or organisations. Transport channels may change over time as systems evolve.

ACP therefore adopts a **defence-in-depth security architecture** that combines multiple complementary mechanisms.

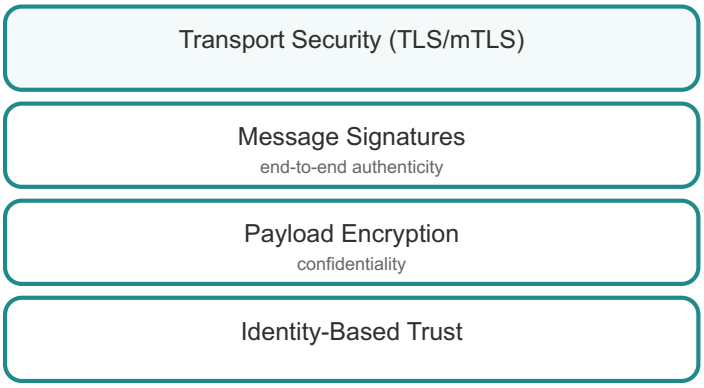


Figure 7.1. ACP layered security model

These mechanisms include:

- transport security
- optional mutual authentication
- protocol-level message signatures
- payload encryption

Each layer protects against a different class of threats.

7.1 The Security Problem in Autonomous Systems

In classical service environments, communication patterns are predictable. A client calls a known API endpoint over HTTPS and receives a response. Trust is largely anchored in infrastructure elements such as certificates or API tokens.

Autonomous systems behave differently.

Agents may initiate communication dynamically. Messages may be routed across several infrastructures before reaching their destination. Participants may operate in different organisational domains. As the Preface explains, machine collaboration environments introduce a trust problem that cannot be solved solely through endpoint-level security.

Because of this, ACP must establish **end-to-end trust** that survives infrastructure changes and routing intermediaries.

7.2 Transport Security

The first layer of protection typically comes from the transport channel itself.

When agents communicate over HTTPS, the TLS protocol provides several important protections:

- confidentiality of the communication channel
- protection against network-level interception
- verification of server identity via certificates

Transport security, therefore, answers the question:

Is the network path secure?

However, transport security alone cannot guarantee the authenticity of messages in a complex distributed architecture.

A message may pass through several infrastructure components that terminate and re-establish TLS connections.

```
Agent A → Gateway → Relay → Broker → Agent B
```

ACP therefore introduces additional security layers at the protocol level.

7.3 Message Signatures

Every ACP message may be signed using the sender's private key. The recipient verifies the signature using the public key contained in the sender's identity document.

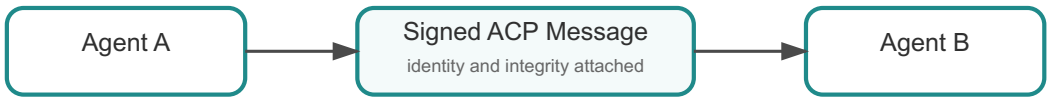


Figure 7.2. Message signature verification

This verification ensures:

1. **Authenticity** — the message originated from the claimed agent
2. **Integrity** — the message contents have not been altered in transit

Because the signature is attached to the message envelope rather than the transport channel, it remains valid even if the message travels through several intermediate systems.

7.4 Payload Encryption

In some environments, confidentiality must extend beyond the transport layer.

For example, a relay may forward messages between organisations without being permitted to read their contents. ACP supports **payload encryption** using the recipient's public key.

Only the intended recipient can decrypt the payload.

```
Agent A → Encrypted Payload → Relay → Agent B
```

This allows ACP messages to traverse infrastructure that may not be fully trusted while preserving confidentiality.

7.5 Mutual TLS

Enterprise environments frequently require stronger authentication mechanisms between communicating systems. Mutual TLS (mTLS) extends TLS so both sides present certificates and verify each other.

ACP integrates with mTLS by treating it as part of the transport layer rather than redefining it inside the protocol. This ensures compatibility with existing enterprise security infrastructure.

7.6 Trust Boundaries and Relays

Relay infrastructure introduces additional trust considerations.

Because relays forward messages between agents, they operate within clearly defined trust boundaries.

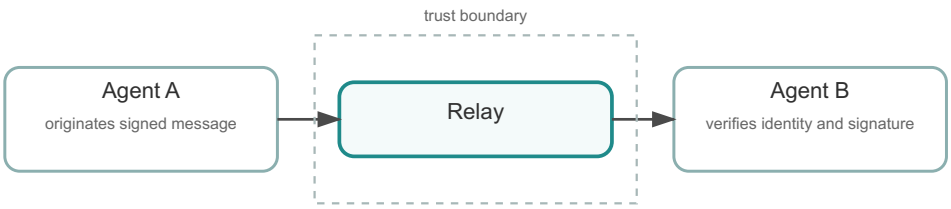


Figure 7.3. Relay trust boundary

ACP ensures that:

- message signatures remain valid across relay hops
- encrypted payloads cannot be inspected by relays
- identity verification occurs at the receiving agent

This allows relay networks to scale across organisations without requiring complete trust in every intermediate system.

7.7 Enterprise Security Integration

ACP's layered security architecture integrates naturally with enterprise security frameworks such as:

- PKI certificate authorities
- secret management platforms (Vault, KMS)
- identity governance systems
- audit and compliance infrastructure

Because ACP identity and message verification operate at the protocol level, organisations can enforce security policies without modifying application logic.

7.8 Defence in Depth

Security in ACP follows the classic **defence-in-depth** principle.

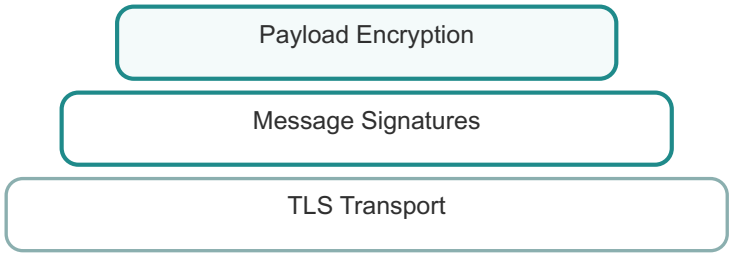


Figure 7.4. Security defence layers

Each layer addresses different risks:

- TLS protects the network channel
- signatures protect authenticity and integrity
- encryption protects confidentiality

Even if one layer fails or is misconfigured, other layers continue to provide protection.

7.9 Summary

ACP's layered security model reflects the realities of autonomous system collaboration.

Transport security protects the network channel, while protocol-level signatures ensure message authenticity and integrity. Payload encryption protects confidentiality even when messages traverse infrastructure that cannot be fully trusted.

Together, these mechanisms create a flexible security architecture capable of supporting both lightweight development environments and highly regulated enterprise deployments.

Chapter 8 - Relay Architecture

In an idealised distributed system, every participant could communicate directly with every other participant. In practice, this assumption rarely holds. Real networks are segmented by firewalls, organisational boundaries, and security policies. Many systems operate in environments where inbound connectivity is restricted or heavily controlled.

The Preface explains that autonomous systems introduce a new class of collaboration problems where machines must coordinate across infrastructure environments and organisational domains. In such ecosystems, direct connectivity cannot always be assumed.

ACP addresses this reality through the concept of a **relay**.

A relay is an intermediary infrastructure component responsible for forwarding ACP message envelopes between agents. Unlike traditional messaging middleware, relays do not interpret message semantics. They route envelopes according to protocol metadata while leaving payload interpretation entirely to the agents.

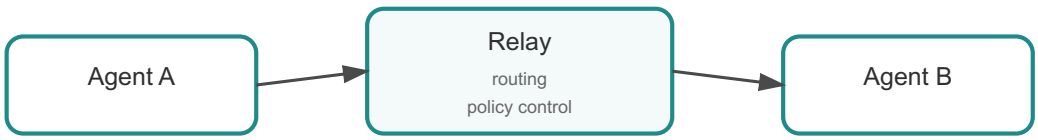


Figure 8.1. Relay routing model

8.1 Why Relays Exist

The need for relays becomes clearer when examining real infrastructure environments.

Agents frequently operate within:

- private enterprise networks
- cloud environments behind load balancers
- partner organisations with restricted inbound connectivity
- research environments separated by network boundaries

Direct communication between every pair of agents would require complex firewall rules and routing policies.

Relays simplify this architecture by acting as communication hubs.

```
Agent A → Relay → Agent B
```

Each agent only needs connectivity to the relay rather than to every other participant in the system.

8.2 Routing Through Relays

Relays route ACP messages based on metadata contained in the envelope.

Because the envelope already includes:

- sender identity
- capability semantics
- cryptographic signatures

the relay does not need to understand or interpret the application payload.

Its role is purely infrastructural: forward messages according to routing hints.

This separation preserves one of ACP's core architectural principles:

infrastructure moves messages; agents interpret them.

8.3 Stateless vs Stateful Relays

Relays can be implemented using different operational models depending on system requirements.

8.3.1 Stateless Relays

Stateless relays forward messages without maintaining long-term knowledge about participants.

Advantages:

- simple architecture
- easy horizontal scaling
- minimal operational complexity

Stateless relays rely on routing hints contained directly in the message envelope.

8.3.2 Stateful Relays

Stateful relays maintain additional routing information, such as:

- agent registrations
- routing tables
- message delivery history

Advantages:

- more efficient routing
- integration with discovery services
- enhanced monitoring and observability

Figure 8.2. Relay architecture types

Both approaches can coexist within the same ACP ecosystem.

8.4 Relay Federation

As ACP deployments grow, multiple relays may cooperate in a **relay federation**.

```
Agent A → Relay A → Relay B → Agent B
```

Federated relays allow ACP networks to span:

- multiple cloud regions
- different enterprise networks
- separate organizations

This architecture resembles other large-scale network systems, such as email routing or Internet routing.

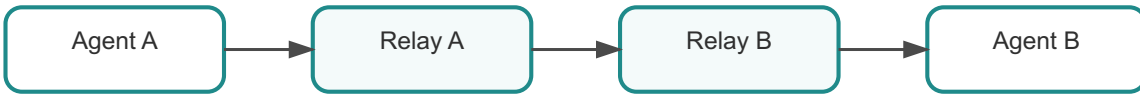


Figure 8.3. Relay federation

8.5 Trust Boundaries

Relays frequently sit at trust boundaries between organisations or network zones.

Because ACP messages include signatures and optional encryption, relays do not need access to the message payload.

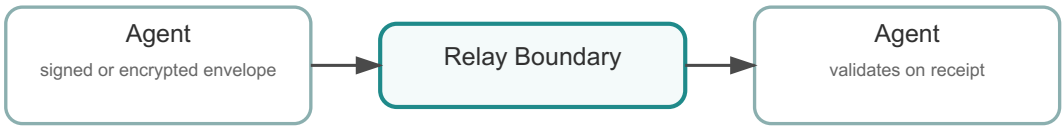


Figure 8.4. Relay trust boundary

This architecture allows organisations to operate a shared relay infrastructure without requiring full trust in every intermediary component.

8.6 Reliability Considerations

Large relay networks must also address reliability.

Typical relay deployments include:

- redundant relay nodes
- load balancing
- health monitoring
- retry mechanisms

Because ACP messages contain unique identifiers, agents can detect duplicate deliveries and maintain consistent semantics even when retries occur.

8.7 Operational Role of Relays

Relays often become an operational focal point in ACP ecosystems.

Operators may use relay infrastructure to:

- monitor traffic patterns
- enforce routing policies
- manage cross-organisation communication
- observe system health and message flow

Because relays operate at the routing layer rather than the application layer, they can provide these capabilities without interfering with business logic.

8.8 Summary

Relays allow ACP to function in real-world network environments where direct connectivity cannot be assumed.

By routing message envelopes rather than interpreting payloads, relays preserve protocol semantics while enabling flexible network architectures.

Relay networks can scale from simple routing hubs to federated infrastructures connecting multiple organisations and trust domains.

Chapter 9 - Capability Negotiation

Autonomous systems evolve over time. New features appear, existing ones change, and different participants may support different subsets of functionality.

This challenge is particularly visible in environments where independent teams deploy software at different cadences. In such ecosystems, it is unrealistic to assume that every participant will upgrade simultaneously. Yet without a mechanism to adapt to these differences, distributed systems quickly become brittle.

ACP addresses this challenge through **capability negotiation**.

Capabilities define the semantic actions that agents can perform or request. Instead of assuming that every agent implements the same interface, ACP allows agents to discover what their peers support and adapt their behaviour accordingly.

This capability-driven interaction model reflects one of the core insights described in the Preface: autonomous systems must be able to collaborate dynamically rather than relying on rigid, preconfigured integrations.

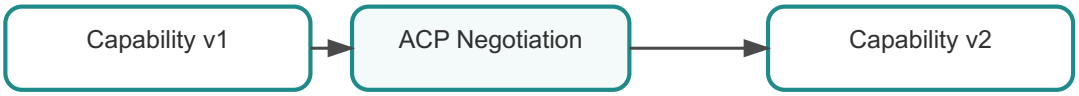


Figure 9.1. Capability negotiation overview

9.1 Capabilities as Semantic Contracts

In traditional API architectures, the semantics of interactions are often embedded implicitly in endpoint structures. A URL such as:

```
POST /execute
```

implicitly defines the action being requested. However, this approach couples semantics tightly to transport mechanisms and endpoint naming conventions.

ACP instead introduces **explicit capability identifiers**.

Examples might include:

- `ping`

- `execute_workflow`
- `challenge`
- `move`
- `request_quote`

Each capability represents a semantic contract between agents.

Because these capabilities are embedded in the protocol rather than the transport, they remain meaningful across HTTP APIs, broker messaging systems, or relay-based routing infrastructures.

9.2 The Evolution Problem

Large distributed ecosystems rarely evolve uniformly.

Different agents may:

- run different software versions
- support different features
- implement domain-specific extensions
- introduce new capabilities over time

Without a negotiation mechanism, introducing new features would require coordinated upgrades across the entire ecosystem. In practice, such coordination is rarely achievable.

Capability negotiation allows systems to evolve incrementally.

Agents that understand a capability can begin using it immediately, while others continue operating using previous interaction patterns.

This model resembles the evolution of Internet protocols such as HTTP, where new features were introduced without breaking existing implementations.

9.3 Discovering Capabilities

Agents must be able to determine which capabilities another agent supports before initiating complex interactions.

ACP supports several discovery mechanisms.

Capabilities may be advertised through:

- identity documents
- discovery registries
- runtime negotiation messages

Each mechanism serves different deployment environments.

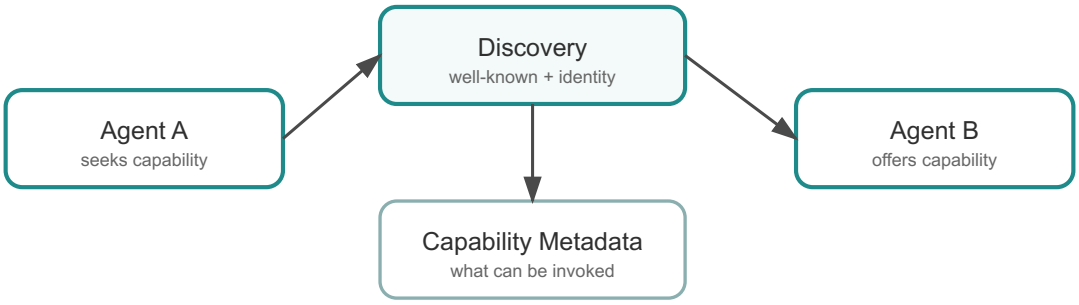


Figure 9.2. Capability discovery

9.3.1 Identity Document Advertisement

In simple environments, capabilities may be included directly in the agent's identity metadata. This allows other agents to discover supported capabilities as part of the identity verification process.

9.3.2 Discovery Registry Advertisement

In larger deployments, discovery registries may maintain capability metadata alongside endpoint information. This allows organisations to maintain centralised visibility of system capabilities.

9.3.3 Runtime Negotiation

In dynamic environments, agents may negotiate capabilities during runtime interactions. An agent can query another agent to determine which capabilities it supports before initiating a workflow.

9.4 Capability Versioning

Capabilities may also evolve over time. A capability might gain additional parameters or support new interaction patterns.

ACP supports versioning by allowing capability identifiers to evolve while maintaining compatibility.

For example:

```
execute_workflow.v1
execute_workflow.v2
```

Agents who understand the newer version can use it, while older agents continue operating with the previous version.

This versioning model prevents ecosystem fragmentation while still allowing innovation.

9.5 Negotiation Workflow

A simplified capability negotiation sequence may look like this:

```
Agent A → Query Capabilities → Agent B
Agent B → Supported Capabilities → Agent A
Agent A → Execute Supported Capability → Agent B
```

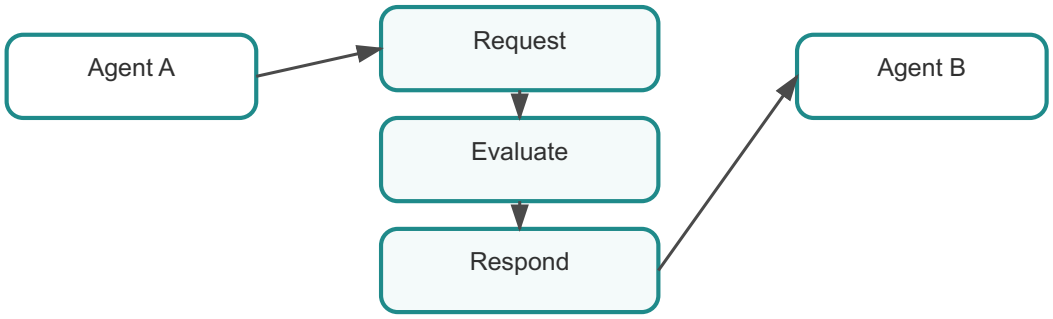


Figure 9.3. Capability negotiation flow

This pattern allows agents to establish compatible interaction patterns before exchanging more complex messages.

9.6 Capability Governance

In enterprise environments, capability negotiation also supports governance.

Organisations may define policies controlling:

- which capabilities particular agents may invoke
- which agents may expose specific capabilities
- which capabilities may cross organisational boundaries

Because capabilities are explicit protocol elements, governance systems can enforce these policies without inspecting application payloads.

9.7 Capability Negotiation in Multi-Agent Ecosystems

Large multi-agent ecosystems may involve hundreds or thousands of participants.

In such environments, capabilities act as the **semantic vocabulary of the ecosystem**.

Agents collaborate by discovering capabilities, negotiating compatible interaction patterns, and exchanging messages that invoke those capabilities.

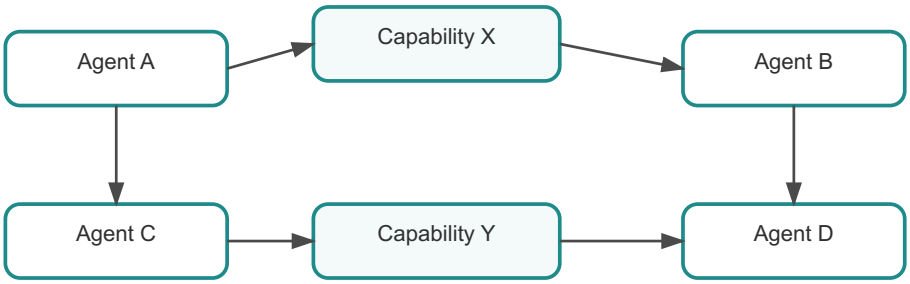


Figure 9.4. Capability ecosystem

This model allows ecosystems to evolve organically while preserving interoperability between participants.

9.8 Summary

Capability negotiation allows ACP ecosystems to evolve without requiring synchronised upgrades across all participants.

By making interaction semantics explicit and negotiable, ACP enables agents to discover supported operations, adapt to version differences, and collaborate dynamically.

This capability-driven model ensures that autonomous systems remain interoperable even as new features and behaviours emerge over time.

Chapter 10 - SDK Architecture

Protocols are defined in specifications, but they become useful only when developers can apply them in real software systems. As discussed earlier in **Chapter 2: Architecture**, the runtime layer is where the protocol becomes executable behaviour.

The Agent Communication Protocol, therefore, relies on a family of **Software Development Kits (SDKs)** that implement the runtime layer of the protocol. These SDKs translate the conceptual model of ACP, identities, messages, discovery, transports, and security, into practical programming interfaces that developers can use to build agents.

This chapter explores how those SDKs are structured, why their architecture matters, and how they maintain interoperability across multiple programming languages.

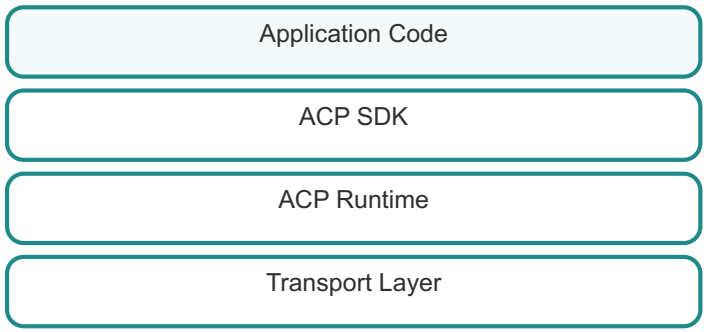


Figure 10.1. ACP SDK architecture

10.1 The Role of SDKs in Protocol Adoption

Protocols rarely succeed based solely on their specifications. History shows that developer experience is one of the strongest predictors of protocol adoption. Technologies such as HTTP, OAuth, and Kubernetes gained widespread adoption not only because of strong design principles but also because developers could easily build software using them.

The same principle applies to ACP.

As the Preface explains, modern autonomous systems require a **collaboration layer** that supports dynamic machine ecosystems. However, that collaboration layer must also be accessible to developers working in different languages, frameworks, and environments.

SDKs provide this accessibility.

They allow developers to interact with the protocol through familiar programming constructs rather than forcing them to manually implement message signing, encryption, discovery resolution, and transport communication.

10.2 Responsibilities of the ACP Runtime

At the heart of every ACP SDK lies the **runtime implementation**.

The runtime is responsible for translating application-level behaviour into protocol interactions. It performs several critical functions:

- constructing ACP message envelopes
- signing outgoing messages with the agent's identity keys
- verifying signatures on incoming messages
- encrypting and decrypting payloads when required
- resolving agent identities through discovery
- selecting the appropriate transport binding

These responsibilities collectively implement the protocol semantics defined in earlier chapters.

Figure 10.2. Runtime responsibilities

Without the runtime layer, each application would need to implement these behaviours independently, a task that would be both error-prone and difficult to maintain.

10.3 Language-Specific SDKs

ACP SDKs are implemented in multiple programming languages, so agents can be built with the tools most appropriate to each environment.

Current implementations include:

- Python
- Java
- Rust
- TypeScript
- Go
- Mojo

Although each SDK uses language-specific constructs, they all implement the same underlying protocol semantics.

For example:

- Python SDKs may use decorators and asynchronous event loops to register capabilities.
- Java SDKs may rely on annotation-based frameworks and dependency injection.
- Rust SDKs often expose strongly typed interfaces built around traits and ownership models.
- TypeScript SDKs integrate naturally with Node.js frameworks and asynchronous programming patterns.

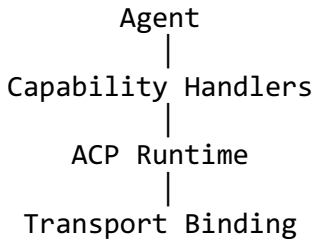
Despite these differences, the behaviour of the protocol remains consistent across languages.

10.4 SDK Interaction Model

Most ACP SDKs expose a simple interaction model for developers.

A typical agent implementation follows three steps:

1. Create an agent instance with an identity.
2. Register capability handlers.
3. Start the runtime loop.



The runtime handles protocol details automatically, allowing developers to focus on application logic.

10.5 Cross-Language Interoperability

One of the key design goals of ACP's SDK architecture is **cross-language interoperability**.

An agent written in Rust must be able to communicate seamlessly with agents written in Python, Java, or TypeScript. Achieving this interoperability requires strict adherence to the protocol specification across all SDK implementations.

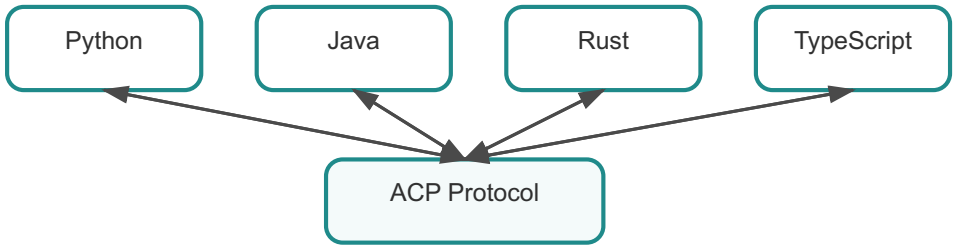


Figure 10.3. Cross-language interoperability

Each SDK therefore shares a common set of responsibilities:

- identical message envelope structure
- identical signature verification logic
- identical capability invocation semantics
- consistent transport binding behaviour

This consistency ensures that agents can collaborate regardless of the programming language used to implement them.

10.6 SDK Design Principles

The ACP SDK ecosystem follows several design principles.

- **Protocol fidelity**
  SDKs must faithfully implement the protocol specification so that agents remain interoperable across languages.

- **Developer ergonomics**
  APIs should feel natural within each programming language. Developers should interact with ACP concepts rather than low-level protocol mechanics.

- **Extensibility**
  SDKs must support new capabilities, transports, and security features without breaking existing applications.

- **Operational transparency**
  Developers and operators should be able to inspect message flows, debugging information, and protocol interactions.

These principles help ensure that the SDK layer remains both technically robust and developer-friendly.

10.7 The Runtime as Collaboration Infrastructure

In small examples, the ACP runtime appears to be a simple library embedded in an application. In large deployments, however, the runtime becomes part of a broader collaboration infrastructure.

When hundreds or thousands of agents operate in the same ecosystem, the runtime layer ensures that each participant adheres to the same communication rules.

This consistency is essential for large-scale interoperability.

It allows different teams, organisations, and programming environments to participate in a shared collaboration protocol without requiring tight coordination between development teams.

10.8 Summary

ACP SDKs translate the protocol specification into practical programming interfaces.

They implement message construction, identity verification, encryption, discovery resolution, and transport communication while allowing developers to focus on business logic.

By maintaining consistent protocol semantics across Python, Java, Rust, and TypeScript implementations, ACP enables autonomous agents to collaborate across heterogeneous software ecosystems.

In the next chapter, we move from protocol implementation to **operational tooling**, examining how command-line interfaces and management utilities help developers and operators interact with ACP systems in practice.

Part III - Using ACP

Chapter 11: Building Your First ACP Agent

Chapter 12: Overlay Adoption Model

Chapter 13: Framework Overlay Wrappers

Chapter 14: Using the ACP CLI

Chapter 15: Multi-Agent Examples

This part moves from protocol theory into practice, showing how developers build, wrap, inspect, and operate ACP-based agents and tools.

Chapter 11 - Building Your First ACP Agent

As discussed earlier in **Chapter 4: Message Model Deep Dive** and **Chapter 5: Discovery and the Well-Known Endpoint**, ACP messages carry both the semantics of an interaction and the trust required to verify it. The remaining question for a developer is practical:

How do we actually build an agent that participates in this system?

In ACP, an agent is simply a program that hosts an ACP runtime and exposes capabilities through that runtime. The runtime handles identity verification, message construction, signature validation, encryption, and transport delivery.

Developers, therefore, focus on defining **capabilities** and **application logic**, while the SDK handles protocol mechanics.

11.1 The Role of the SDK

ACP SDKs implement the runtime layer described earlier in the architecture chapter.

Typical responsibilities include:

- loading agent identity and cryptographic keys
- constructing ACP message envelopes
- verifying message signatures
- decrypting payloads
- resolving discovery hints
- delivering messages via transports

This abstraction ensures developers interact with **protocol concepts**, not low-level cryptography or networking.

11.2 Minimal Agent Structure

A typical ACP agent follows a simple structure:

1. Load the identity
2. Register capability handlers
3. Start the runtime

11.2.1 Python Example

```python
from acp import Agent

agent = Agent("agent:demo.service")

@agent.route("ping")
def ping(payload):
    return {"status": "ok"}
```

```
agent.run()
```

11.2.2 Rust Example

```rust
let agent = Agent::new("agent:demo.service");

agent.route("ping", |payload| {
    Ok(json!({"status":"ok"}))
});

agent.run();
```

11.2.3 TypeScript Example

```typescript
const agent = new Agent("agent:demo.service")

agent.route("ping", async (payload) => {
  return { status: "ok" }
})

agent.run()
```

11.3 Agent Lifecycle

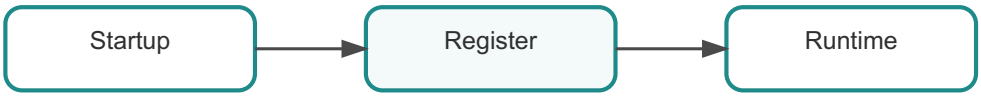


Figure 11.1. Agent lifecycle

An ACP agent typically follows this lifecycle:

1. Startup and identity loading
2. Capability registration
3. Message processing loop
4. Shutdown or redeployment

The runtime manages protocol details, so developers focus on application behaviour.

11.4 Message Flow

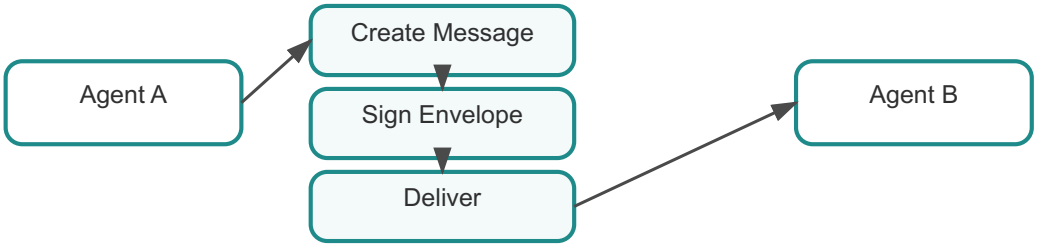


Figure 11.2. Message flow

Interaction between two agents typically follows:

Agent A → create message
Agent A → sign envelope
Agent A → send message
Agent B → verify signature
Agent B → execute capability
Agent B → return response

11.5 Cross-Language Ecosystem

ACP supports multiple SDK implementations, including:

- Python
- Java
- Rust
- TypeScript
- Go
- Mojo

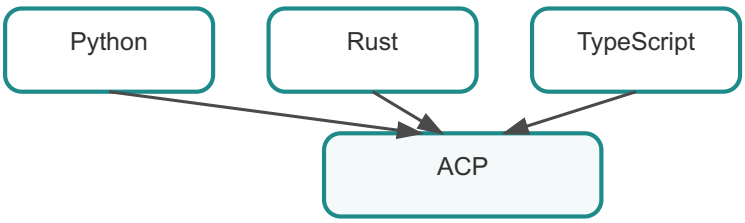


Figure 11.3. SDK ecosystem

Despite language differences, the protocol semantics remain identical.

11.6 Developer Stack

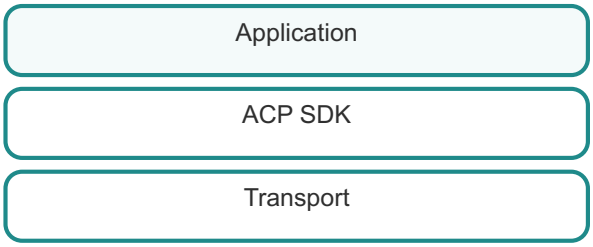


Figure 11.4. Developer stack

ACP SDKs allow developers to build agents without implementing:

- cryptography
- transport negotiation
- discovery resolution

This dramatically lowers the barrier to building interoperable agents.

11.7 Summary

An ACP agent is simply an application that hosts an ACP runtime and exposes capability handlers.

The SDK handles protocol mechanics such as signing, encryption, discovery, and transport delivery. This allows developers to focus on their agents' behaviour rather than the infrastructure required to support them.

Chapter 12 - Overlay Adoption Model

In **Chapter 2: Architecture**, we introduced the layered structure of ACP and noted that the protocol must coexist with existing infrastructure. In practice, this requirement is not merely an architectural convenience; it is essential for adoption. As discussed in the Preface, most organisations already operate complex distributed systems built on HTTP APIs, message brokers, service meshes, and internal governance platforms.

Replacing that infrastructure outright would be unrealistic. Successful protocols historically gain adoption by allowing incremental integration rather than demanding immediate architectural transformation. The Internet itself evolved this way: new capabilities were layered on top of existing infrastructure rather than replacing it all at once.

ACP, therefore, introduces the **overlay adoption model**.

Overlay mode allows ACP semantics to be layered on top of existing services and communication paths without requiring immediate architectural redesign. This capability is one of the key reasons ACP can realistically be adopted within large organisations.

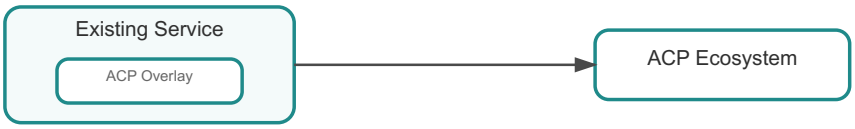


Figure 12.1. Overlay concept

12.1 The Adoption Challenge

Technology adoption rarely happens in isolation. Enterprise systems evolve gradually because existing systems carry operational risk, regulatory obligations, and long-term maintenance commitments.

Architectural changes typically face several constraints:

- infrastructure migration risk
- operational disruption
- integration dependencies
- governance and compliance reviews

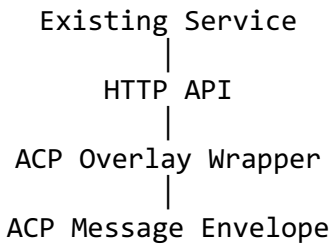
Protocols that require organisations to replace existing infrastructure before realising value often fail to gain traction. The history of distributed systems is filled with technically elegant designs that never achieved adoption because they required too much immediate change.

ACP was deliberately designed to avoid this outcome.

The overlay adoption model recognises that organisations need to introduce new capabilities gradually while maintaining compatibility with existing systems.

12.2 Overlay Mode

In overlay mode, ACP operates as a wrapper around an existing communication interface.



The underlying service continues to function normally. Clients who do not use ACP can continue interacting with the service exactly as before.

The overlay layer adds additional responsibilities:

- verifying ACP message signatures
- decrypting payloads when necessary
- translating ACP envelopes into application calls
- signing and encrypting responses

This architecture allows a service to participate in ACP ecosystems while still maintaining compatibility with traditional API consumers.

12.3 Why Overlay Matters

Overlay mode significantly lowers the barrier to adoption.

Instead of requiring a complete architectural redesign, ACP can be introduced in a small, controlled scope. A team might start by enabling ACP on a single service endpoint. That service immediately gains the ability to participate in secure agent collaboration without disrupting existing workflows.

This incremental approach allows organisations to experiment with ACP in realistic environments before committing to deeper integration.

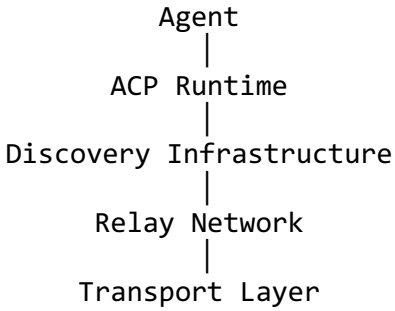
Historically, technologies that enabled incremental adoption, such as REST APIs, OAuth, and container orchestration, spread much faster than those that require wholesale infrastructure replacement.

ACP follows the same pattern.

12.4 Native ACP Mode

Overlay adoption is not the final stage of the architecture. It is an entry point.

As organisations become more comfortable with ACP, they may transition toward **native ACP deployments**.



In native mode, ACP becomes the primary collaboration layer between agents. Instead of wrapping existing services, applications are designed directly around ACP runtimes and message semantics.

Native deployments unlock the full set of ACP capabilities, including:

- dynamic discovery
- relay-based routing
- capability negotiation
- federated agent ecosystems

However, these features are not required for initial adoption.

12.5 Migration Path

A realistic ACP adoption journey often progresses through several stages.

```
Stage 1: Existing services with ACP overlay wrappers
Stage 2: Hybrid systems combining overlay and native agents
Stage 3: Native ACP agent ecosystems
```

During the first stage, ACP may simply provide a secure collaboration layer on top of existing APIs. At the second stage, some services may evolve into native agents while others remain overlay services. Finally, organisations may operate fully native ACP ecosystems.

Because ACP protocol semantics remain consistent across all stages, systems built at each stage remain interoperable.

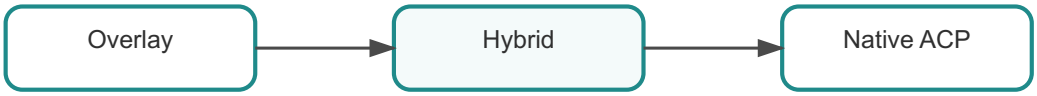


Figure 12.2. Migration path

12.6 Organisational Benefits

The overlay model also provides organisational benefits beyond technical adoption.

Teams can experiment with ACP without requiring organisation-wide approval. Security teams can evaluate the protocol's cryptographic model in isolation. Platform teams can gradually introduce relay infrastructure and discovery services.

This incremental adoption path aligns with real enterprise decision-making processes.

Rather than forcing a single large architectural decision, ACP enables a sequence of smaller, lower-risk improvements.

12.7 Overlay in Multi-Organisation Environments

Overlay adoption is also useful when ACP is introduced across multiple organisations.

Different organisations may adopt ACP at different speeds. Some may deploy full native infrastructures, while others begin with overlay services integrated into existing systems.

Because the protocol semantics remain consistent, these systems can still collaborate.

This flexibility allows ACP ecosystems to grow organically rather than requiring centralised coordination.

12.8 Relationship to the Machine Collaboration Problem

The Preface introduced the **machine collaboration problem**: modern autonomous systems require a shared communication layer that supports identity, trust, discovery, and interoperability.

Overlay adoption helps address this problem in a practical way. Instead of waiting for a fully standardised agent ecosystem to appear, organisations can begin implementing the collaboration layer immediately.

The protocol can evolve gradually while delivering value at every stage of adoption.

12.9 Summary

The overlay adoption model exists because real systems cannot be replaced overnight.

By allowing ACP to wrap existing services, organisations can experiment with the protocol without committing to a full architectural migration. Over time, these systems can evolve toward native ACP deployments that support discovery, relay networks, and dynamic agent collaboration.

This incremental adoption strategy is central to ACP's design. It ensures that the protocol can be introduced pragmatically within real-world infrastructure environments while still enabling the long-term vision of large-scale autonomous system collaboration.

Chapter 13 - Framework Overlay Wrappers

In **Chapter 12: Overlay Adoption Model**, we examined how ACP can be introduced gradually into existing systems without requiring immediate architectural transformation. The overlay approach allows organisations to experiment with ACP semantics while preserving compatibility with existing infrastructure.

However, overlay adoption requires practical integration mechanisms. Modern software systems are rarely built from scratch; instead, they rely on established frameworks such as Flask, FastAPI, Spring Boot, Express, and other web application frameworks. If ACP required developers to abandon these ecosystems, its adoption would be dramatically slower.

Framework overlay wrappers provide the bridge between existing application frameworks and the ACP runtime.

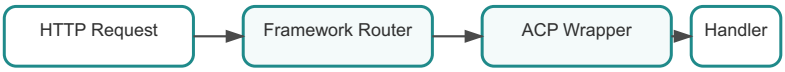


Figure 13.1. Framework overlay architecture

By translating framework-level requests into ACP message envelopes, these wrappers allow applications to participate in ACP communication while preserving the development patterns teams already use.

13.1 The Integration Problem

The Preface describes the **machine collaboration problem**: modern systems require a shared collaboration layer that supports autonomous participants. Yet these participants often exist within existing service frameworks.

For example:

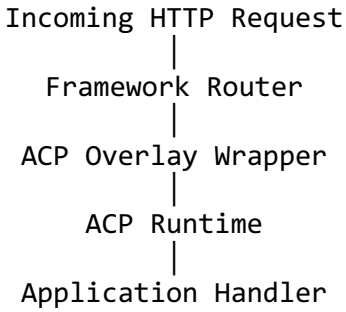
- Python services may run on Flask or FastAPI
- Java applications often rely on Spring Boot
- Node.js systems typically use Express or similar frameworks

These frameworks already define routing, request handling, dependency injection, and lifecycle management patterns. Asking developers to replace these frameworks with a completely new runtime environment would create significant friction.

Overlay wrappers solve this problem by adapting the ACP runtime to the conventions of these frameworks.

13.2 Wrapper Architecture

At a conceptual level, an overlay wrapper sits between the application framework and the ACP runtime.



The wrapper performs several critical functions:

- extracts the ACP envelope from the incoming request
- verifies message signatures and identity metadata
- decrypts the payload if encryption is used
- passes the payload to the application handler
- signs and encrypts the response before returning it

The application developer, therefore, interacts with simple application objects while the wrapper handles protocol mechanics.

13.3 Python Framework Integration

Python ecosystems frequently use lightweight frameworks such as Flask or FastAPI. These frameworks rely on decorators or route definitions to bind HTTP paths to application functions.

An ACP overlay wrapper can integrate directly with this model.

Example Flask integration:

```
@app.route("/task")
@acp_overlay_inbound(config)
def task(payload):
    return execute_task(payload)
```

When a request arrives, the wrapper intercepts the call before it reaches the application function.

The wrapper then:

1. parses the ACP envelope
2. verifies the message signature
3. decrypts the payload if required

4. invokes the handler
5. signs and encrypts the response

The developer does not need to implement any protocol-specific logic.

13.4 Java Framework Integration

Java applications often use Spring Boot, which provides annotation-based request routing and dependency injection. ACP integrates with this model through runtime helpers that wrap controller methods.

Example Spring controller integration:

```
@PostMapping("/task")
public ResponseEntity<?> task(HttpServletRequest request) {
    return OverlayHttpRuntime.handle(request, payload -> {
        return processTask(payload);
    });
}
```

Here, the `OverlayHttpRuntime` helper performs the protocol translation. The application logic remains focused on domain behaviour rather than message validation or cryptographic operations.

13.5 TypeScript and Node.js Integration

Node.js ecosystems often rely on Express-style routing patterns. ACP wrappers can integrate with these patterns in a similar way.

Example Express integration:

```
app.post("/task", acpOverlay(async (payload) => {
  return processTask(payload)
}))
```

Again, the wrapper layer automatically performs envelope parsing, signature verification, and response signing.

13.6 Why Framework Wrappers Matter

Framework wrappers play a critical role in ACP adoption.

Historically, protocols that required developers to abandon existing frameworks have struggled to gain traction. Developers prefer technologies that integrate smoothly with tools they already understand.

Overlay wrappers allow ACP to coexist with existing frameworks rather than replacing them.

This design aligns directly with the Preface's emphasis on **low-friction adoption**. Organizations can experiment with ACP in isolated services before deciding whether to expand its use across larger systems.

13.7 Operational Implications

Overlay wrappers also simplify operational adoption.

Platform teams can introduce ACP support at the framework layer without requiring application teams to redesign their codebases. Security teams can evaluate ACP identity and message verification mechanisms independently of business logic.

This separation reduces the risk associated with introducing new communication protocols into existing systems.

13.8 Wrapper Design Principles

ACP framework wrappers follow several design principles.

Transparency
The wrapper should not change the application programming model.

Protocol correctness
All envelope validation, signature verification, and encryption handling must occur before the application handler executes.

Framework compatibility
Wrappers must integrate naturally with framework routing and lifecycle mechanisms.

Minimal developer friction
Developers should write application logic exactly as they would in a non-ACP service.

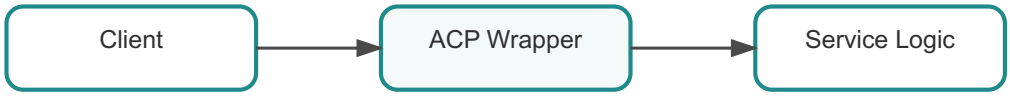


Figure 13.2. Wrapper interaction model

These principles ensure that ACP can be adopted without disrupting developer workflows.

13.9 Toward Native ACP Services

Framework wrappers represent an important transitional step. They allow existing services to participate in ACP communication while maintaining compatibility with traditional API clients.

Over time, however, some systems may transition toward **native ACP agents** where the runtime becomes the primary interaction layer.

In such architectures, overlay wrappers may disappear entirely as applications are designed directly around the ACP runtime.

This transition reflects the gradual evolution described in Chapter 12.

13.10 Summary

Framework overlay wrappers enable ACP to integrate with existing application frameworks such as Flask, FastAPI, Spring Boot, and Express.

By translating framework requests into ACP messages, these wrappers allow applications to participate in secure agent communication without abandoning familiar development patterns.

This capability dramatically lowers the barrier to adoption and allows organisations to introduce ACP gradually while preserving compatibility with existing systems.

Chapter 14 - Using the ACP CLI

While SDKs allow applications to participate in the protocol, developers and operators often need tools for interacting with ACP systems directly.

The ACP command-line interface (CLI) fulfils this role.

The CLI provides utilities for:

- creating identities
- discovering agents
- sending test messages
- inspecting relay infrastructure

These capabilities make the CLI an important companion to the SDKs.

14.1 Why Operational Tooling Matters

As explained in the Preface, autonomous systems operate in dynamic environments where participants may change location, capabilities evolve, and infrastructure may span multiple organisational boundaries.

Operational tooling becomes critical in such environments because engineers must be able to:

- inspect system state
- debug message flows
- verify identity and trust relationships
- test interactions without writing application code

The ACP CLI provides these capabilities in a simple, scriptable interface.

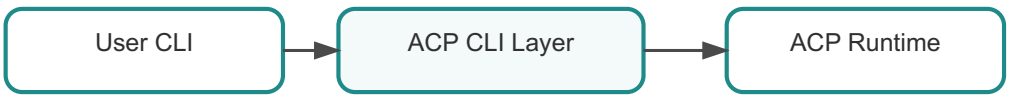


Figure 14.1. CLI architecture

14.2 Creating an Identity

A new identity can be generated with:

```
acp identity create
```

This command creates:

- an identity document
- associated signing keys
- encryption keys if required

The resulting identity can then be used by an agent runtime.

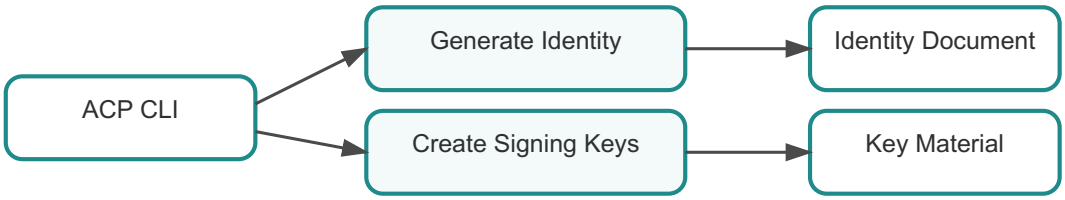


Figure 14.2. Identity creation flow

14.3 Discovering Agents

The CLI can query the well-known endpoint of a remote agent:

```
acp discover well-known https://agent.example.com
```

This command retrieves the metadata described in Chapter 5 and displays the transport hints needed for communication.

14.4 Sending Test Messages

Developers often need to test an agent without writing a full application.

The CLI supports direct message sending:

```
acp message send agent:demo.service ping
```

The CLI constructs the envelope, signs the message, and delivers it via the appropriate transport.

Figure 14.3. Message debug flow

14.5 Inspecting Relay Infrastructure

Operators can also use the CLI to inspect relay state.

```
acp relay status
```

Commands like this reveal routing tables, connected agents, and relay health.

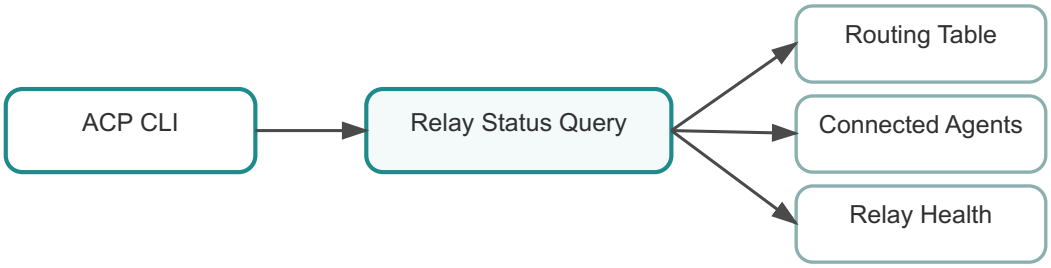


Figure 14.4. Relay inspection

14.6 Operational Stack

In larger environments, the CLI becomes part of the operational toolchain.

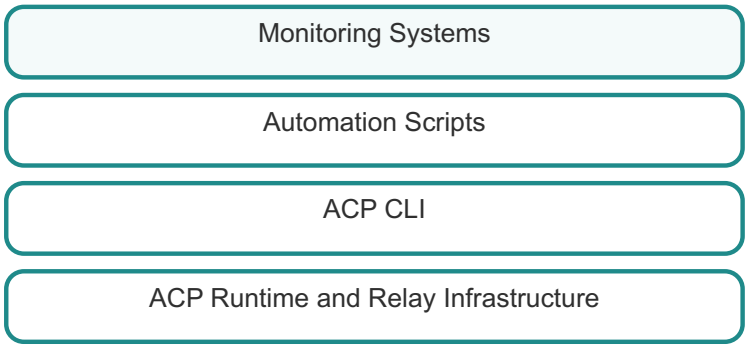


Figure 14.5. Operational stack

The CLI is often used alongside:

- monitoring systems
- automation scripts
- CI/CD pipelines
- debugging tools

14.7 Summary

The ACP CLI provides operational visibility and testing capabilities for developers and system operators.

By exposing identity management, discovery, message sending, and relay inspection tools, the CLI complements the SDKs and simplifies debugging.

Chapter 15 - Multi-Agent Examples

Concepts and architecture become easier to understand when seen in action. Throughout the previous chapters, we examined the components that make the Agent Communication Protocol possible: identity, message envelopes, discovery, transport independence, relays, capability negotiation, and developer tooling.

However, protocols are ultimately validated through **real interactions**. The most effective way to understand ACP is to observe autonomous agents collaborating in realistic scenarios.

This chapter presents two reference systems used throughout ACP development:

- a **distributed chess system**
- a **multi-agent poker system**

These examples demonstrate how the protocol behaves in practice and highlight several important properties of ACP.

As the Preface explains, modern autonomous systems must cooperate dynamically in environments where infrastructure is heterogeneous and constantly evolving. These examples illustrate how ACP provides the collaboration layer required to make such cooperation practical.

15.1 Why Examples Matter

Distributed system architecture often appears clear when presented through diagrams and theoretical models. Yet the real challenges only become visible when systems interact under realistic conditions.

Examples reveal how:

- messages are exchanged between agents
- capabilities drive interaction patterns
- discovery resolves identities dynamically
- relays route communication across network boundaries

The examples in this chapter intentionally use **simple application domains** so that the reader can focus on the protocol mechanics rather than the complexity of the domain itself.

The chess system demonstrates direct agent-to-agent collaboration, while the poker system illustrates multi-participant coordination.

15.2 Chess Agents

In the chess example, each player is represented by an autonomous ACP agent. Two agents communicate directly using ACP messages to conduct a game.

The core capabilities exposed by a chess agent include:

- `challenge`
- `accept`
- `move`
- `game_state`

A simplified interaction flow looks like this:

```
Player A → challenge → Player B
Player B → accept → Player A
Player A → move → Player B
Player B → move → Player A
```

Each step corresponds to a structured ACP message.

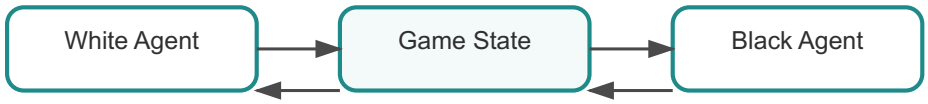


Figure 15.1. Chess interaction

Because the ACP runtime automatically handles message envelopes, signatures and transport bindings, the chess application logic remains surprisingly simple.

The agent focuses only on domain logic, such as validating moves and updating game state.

15.2.1 Message Flow in the Chess Example

When Player A challenges Player B, the following steps occur:

1. Player A constructs a message envelope.
2. The runtime signs the message with Player A's identity key.
3. The message is delivered via the configured transport.
4. Player B verifies the signature.
5. Player B executes the `challenge` capability handler.
6. Player B returns an acceptance response.

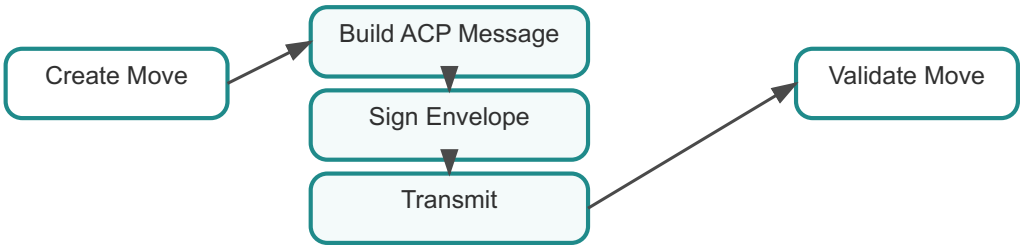


Figure 15.2. Chess message lifecycle

This interaction demonstrates several protocol properties:

- identity verification
- message integrity validation
- capability invocation
- transport-independent delivery

All of these behaviours occur automatically within the runtime layer described in **Chapter 10: SDK Architecture**.

15.3 Poker Agents

The poker example demonstrates a more complex scenario involving multiple agents collaborating simultaneously.

In this system:

- several player agents participate in the game
- a dealer agent coordinates gameplay
- players submit decisions through ACP messages

```
Player A
Player B → Dealer → Game State
Player C
Player D
```

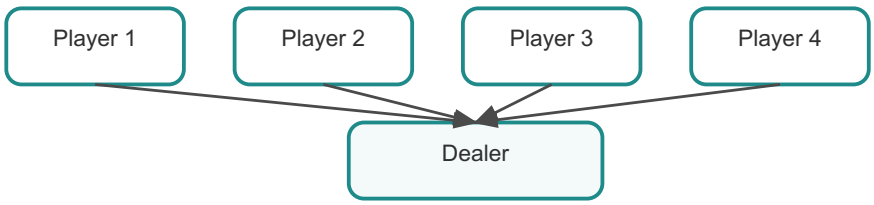


Figure 15.3. Poker architecture

The dealer acts as an orchestrator that receives player actions, updates the game state, and broadcasts results to all participants.

This pattern is common in many distributed systems where one agent coordinates activity among several participants.

15.3.1 Many-to-Many Communication

The poker example highlights an important feature of ACP: support for **many-to-many communication patterns**.

Traditional service architectures often assume a simple request-response interaction between two systems. Autonomous systems frequently require more complex coordination patterns.

ACP messages enable patterns such as:

- broadcast updates
- coordinated decision making
- multi-step workflows
- negotiation between participants

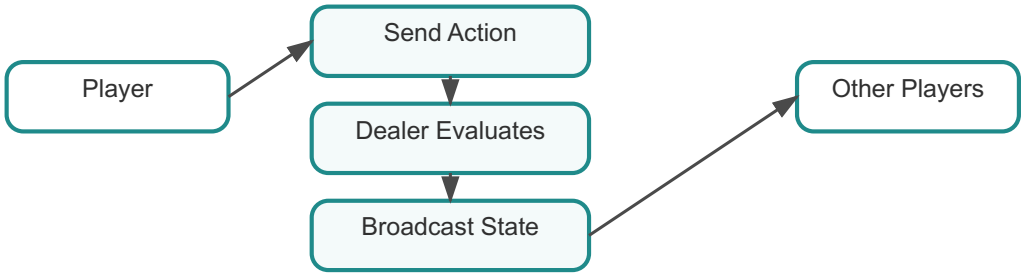


Figure 15.4. Multi-agent coordination

These patterns are common in real applications such as supply-chain systems, trading platforms, and distributed robotics coordination.

15.3.2 Cross-Language Interoperability

Another important property demonstrated by the examples is interoperability between SDK implementations.

The chess and poker agents have been implemented using different languages:

- Python agents
- Rust agents
- Go agents
- TypeScript agents
- Java agents

Because the protocol semantics remain consistent across SDK implementations, these agents can interact seamlessly.

This property is critical for large ecosystems where different teams choose different technology stacks.

15.4 Discovery in Action

The examples also illustrate the discovery mechanisms described earlier in the book.

Agents do not need to know the infrastructure location of their peers in advance. Instead, they resolve identities through discovery mechanisms such as:

- well-known endpoints
- discovery registries
- relay routing hints

This capability allows agents to find each other dynamically as systems evolve.

15.5 Lessons from the Examples

Although the chess and poker demonstrations are simple, they highlight several key characteristics of ACP.

First, **protocol semantics remain stable regardless of transport**. Messages may travel through HTTP APIs, broker systems, or relay networks without changing their meaning.

Second, **identity and message signatures provide end-to-end trust**. Even when messages traverse multiple intermediaries, the receiving agent can verify the sender.

Third, **capabilities enable flexible interaction patterns**. Agents negotiate actions through semantic identifiers rather than rigid endpoint conventions.

These properties together form the foundation of scalable autonomous system collaboration.

15.6 Summary

The chess and poker examples demonstrate how ACP operates in practice. By observing agents exchanging messages, negotiating capabilities, and coordinating behaviour, we see how the protocol transforms abstract concepts into working collaboration systems.

These demonstrations illustrate that ACP is not merely a theoretical architecture. It is a practical protocol capable of supporting real multi-agent ecosystems.

In the next chapters, we move from examples toward **operational deployment patterns**, examining how ACP infrastructures operate in production environments.

Part IV - ACP Deployment and Governance

Chapter 16: Operating ACP Systems

Chapter 17: Relay Federation and Network Topology

Chapter 18: Enterprise Deployment Patterns

Chapter 19: Cross-organisation Collaboration

This part covers large-scale deployment, relay federation, enterprise integration, operational governance, and cross-organisation collaboration.

Chapter 16 - Operating ACP Systems

Earlier chapters explained the conceptual and technical structure of ACP: identity, message envelopes, discovery, transports, relays, and SDK implementations. Those chapters answered the question *how the protocol works*. For enterprise architects and operations teams, however, a different question matters just as much:

How does ACP behave when it becomes part of a real production system?

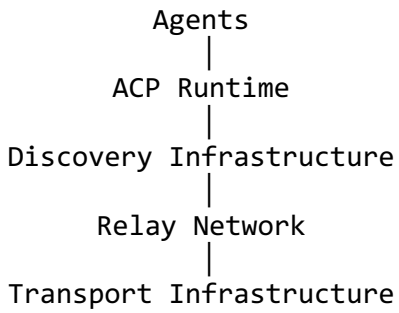
Operating ACP systems requires thinking of the protocol not merely as a developer tool, but as a **collaboration platform embedded within operational infrastructure**. This shift in perspective mirrors the transition described in the Preface: modern distributed systems are evolving into ecosystems of autonomous participants that interact continuously across heterogeneous environments.

In these environments, reliability, observability, governance, and operational safety become just as important as protocol semantics.

16.1 From Protocol to Operational Platform

When developers first encounter ACP, they often view it as a messaging protocol or communication library. In production environments, the picture becomes broader.

ACP deployments typically consist of multiple cooperating components:



Each layer carries operational responsibilities.

- **Agents** implement business logic.
- **Runtimes** enforce protocol semantics and cryptographic verification.
- **Discovery services** map identities to reachable endpoints.
- **Relays** route messages across network boundaries.
- **Transport layers** provide the underlying communication channels.

Understanding the operational behaviour of each layer is essential when deploying ACP in real environments.

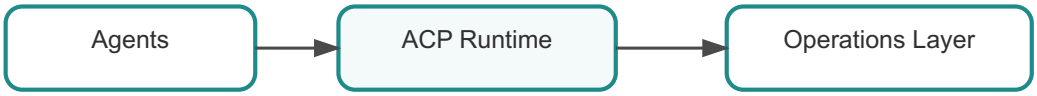


Figure 16.1. ACP operational layers

16.2 Observability in Distributed Agent Systems

One of the most difficult problems in distributed systems is visibility. Traditional architectures often rely on logs or metrics generated by individual services, but reconstructing interactions across multiple services can be difficult.

ACP improves observability by including structured metadata in every message.

Each envelope includes identifiers such as:

- `message_id`
- `agent_id`
- `timestamp`
- `capability`

These fields allow monitoring systems to reconstruct the path of a message through the system.

```
Agent A
   | message_id:8472
   ▼
 Relay
   |
   ▼
Agent B
```

This capability resembles the distributed tracing systems used in modern microservice architectures. Instead of inferring relationships from logs, operators can follow message identifiers directly.

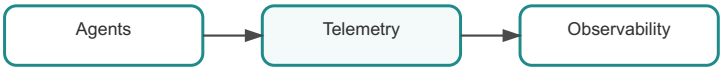


Figure 16.2. Message trace model

This dramatically simplifies debugging in large agent ecosystems.

16.3 Operational Failure Modes

Real systems fail in predictable ways. ACP deployments must account for these conditions from the beginning.

Common operational issues include:

- stale discovery entries
- temporary transport failures
- relay outages
- duplicate message deliveries
- capability mismatches between agents

ACP addresses these scenarios through protocol features described in earlier chapters.

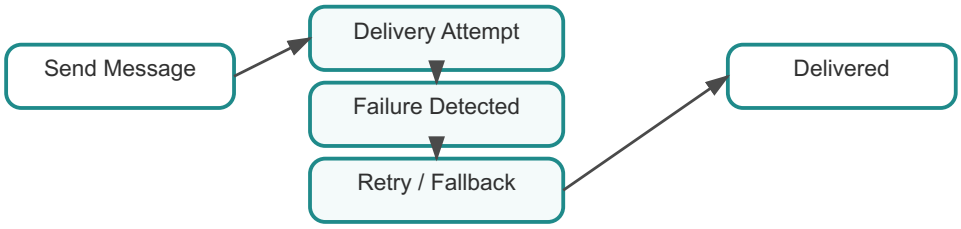


Figure 16.3. Relay operations model

For example:

- message identifiers allow duplicate detection
- discovery refresh mechanisms update endpoint mappings
- capability negotiation allows systems to adapt to version differences
- relay federation allows routing to bypass failing infrastructure

These mechanisms work together to provide resilience across the entire collaboration ecosystem.

16.4 Relay Operations

Relays often become central operational infrastructure in ACP deployments.

Unlike application agents, relays are responsible for routing rather than business logic. Their operational characteristics, therefore, resemble those of network infrastructure.

Typical relay deployment patterns include:

- horizontally scaled relay clusters
- health monitoring and automatic failover
- rate limiting and routing policies
- federation across organisational domains

Because relays do not interpret message payloads, they can scale independently from application logic while preserving protocol semantics.

16.5 Discovery Infrastructure Operations

Discovery services provide another operational component.

In small deployments, discovery may be static or based on well-known endpoints. Large ecosystems often rely on discovery registries.

Operators must consider:

- registry availability
- identity trust policies
- update propagation latency
- registry federation across organisations

Discovery infrastructure becomes particularly important in environments where agents frequently move between infrastructure locations.

16.6 Operational Governance

Enterprise environments require governance mechanisms that extend beyond technical reliability.

Organisations must control:

- which agents are trusted
- which capabilities may cross organisational boundaries
- which relay networks are authorised for communication
- how interactions are audited

Because ACP embeds identity and capabilities directly into messages, governance policies can be expressed at the protocol level rather than through ad-hoc infrastructure rules.

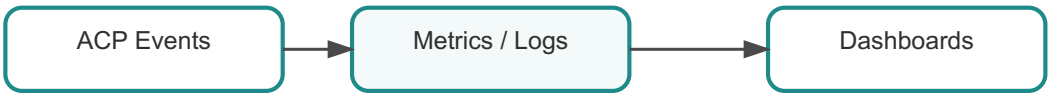


Figure 16.4. Governance observability

This property makes ACP particularly attractive for regulated environments such as financial systems or healthcare networks.

16.7 Security Monitoring

Security monitoring in ACP systems relies on several signals:

- signature verification failures
- unexpected identity claims
- capability misuse patterns
- abnormal message traffic

Because the protocol separates identity from infrastructure location, security systems can track behaviour at the **participant level** rather than the infrastructure level.

This capability aligns with modern zero-trust security architectures, which treats identity rather than network location as the primary security primitive.

16.8 Automation and Tooling

Operational teams rarely manage distributed systems manually. Automation tools play a central role in monitoring and maintenance.

ACP ecosystems often integrate with:

- observability platforms (Prometheus, OpenTelemetry)
- infrastructure orchestration systems
- CI/CD pipelines
- security monitoring tools

The ACP CLI described in Chapter 14 becomes particularly valuable in this context because it allows operators to interact directly with the protocol components without writing application code.

16.9 Lessons from Large Distributed Systems

Many of the operational challenges faced by ACP deployments are not unique. They echo lessons learned from earlier distributed systems, such as:

- large-scale messaging infrastructures
- microservice platforms
- Internet routing systems

The key difference is that ACP systems treat **agents as autonomous participants** rather than static services.

This distinction changes how systems must be monitored and governed.

Operational infrastructure must support dynamic participants, evolving capabilities, and federated collaboration environments.

16.10 Summary

Operating ACP systems requires treating the protocol as an **operational platform** rather than simply a communication library.

Agents, runtimes, discovery infrastructure, relay networks, and transport layers together form a distributed collaboration environment.

The structured nature of ACP messages provides powerful observability and debugging capabilities, while the relay infrastructure and discovery services enable scalable routing and coordination.

When deployed thoughtfully, ACP can support large-scale autonomous ecosystems while maintaining reliability, security, and governance across the organisation boundaries.

Chapter 17 - Relay Federation and Network Topology

In **Chapter 8: Relay Architecture** we introduced relays as routing infrastructure for ACP messages. Relays allow agents to communicate even when direct connectivity is impractical or impossible. However, large real-world environments rarely consist of a single relay operating inside a single network boundary. Instead, they are composed of multiple infrastructure domains, each with its own security policies, operational practices, and governance requirements.

This reality becomes particularly important when ACP systems are deployed across multiple organisations, cloud regions, or regulatory environments. In such scenarios, the relay layer must evolve beyond a single routing component into a **federated network topology**.

The idea of relay federation is inspired by earlier large-scale network systems. Email infrastructure, for example, relies on federated mail transfer agents that forward messages across domains. Internet routing protocols allow independent networks to cooperate while maintaining their own governance. ACP adopts similar architectural principles to enable collaboration across autonomous machine ecosystems.

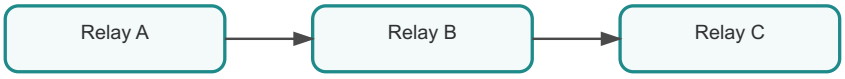


Figure 17.1. Relay federation overview

17.1 The Need for Federation

Modern enterprise infrastructures are inherently segmented. Security controls, organisational boundaries, and regulatory requirements frequently prevent unrestricted communication between systems.

Typical constraints include:

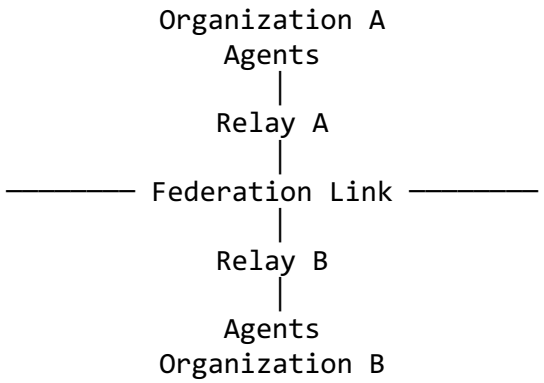
- firewall segmentation between internal networks
- regional infrastructure isolation
- regulatory separation of data environments
- partner network gateways
- zero-trust security policies

These constraints mean that two agents belonging to different organisations often cannot communicate directly, even if they share a common protocol.

As the Preface describes, autonomous ecosystems require a collaboration layer capable of operating across heterogeneous infrastructure environments. Relay federation provides the mechanism that allows ACP to operate under these conditions.

17.2 Basic Federation Model

At its simplest, relay federation connects two relay networks belonging to different organisations.



Each organisation operates its own relay infrastructure and maintains independent governance policies.

The federation link allows messages to move between the two relay networks when permitted.

This architecture allows organisations to collaborate without requiring either side to relinquish control over its internal infrastructure.

17.3 Routing Across Federation Boundaries

When a message crosses a federation boundary, several routing steps occur:

1. The originating agent sends the ACP message to its local relay.
2. The local relay determines that the destination agent belongs to a remote network.
3. The relay forwards the message to the federated peer relay.
4. The receiving relay delivers the message to the destination agent.

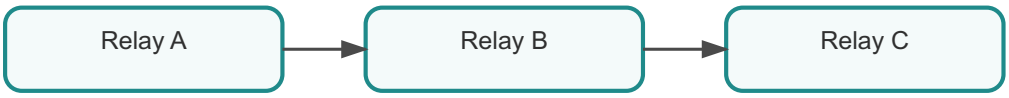


Figure 17.2. Federated routing path

Because ACP messages include signed envelopes and optional encryption, the relays do not need to interpret the message contents.

Their responsibility is limited to routing the message according to protocol metadata and federation policy.

17.4 Policy Enforcement in Federated Networks

Federation introduces governance challenges that do not appear in single-organisation deployments.

Organisations must maintain control over which external systems are allowed to interact with their agents.

Typical federation policies may include:

- identity allowlists or deny lists
- capability restrictions for cross-domain interactions
- routing constraints limiting which relays may forward traffic
- rate limits for external participants

ACP messages carry structured metadata such as agent identities and capability names. This allows relays to enforce many of these policies without decrypting message payloads.

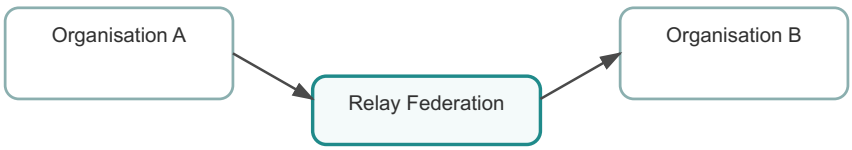


Figure 17.3. Federation policy enforcement

This design preserves confidentiality while still allowing administrative control.

17.5 Reliability in Federated Relay Networks

Large distributed systems must be resilient to infrastructure failures. Relay federation, therefore, typically incorporates redundancy mechanisms.

Common strategies include:

- multiple relay instances per region
- health checks and failover routing
- load balancing between relays
- alternate federation paths

If one relay becomes unavailable, messages can be rerouted through another relay within the federation.

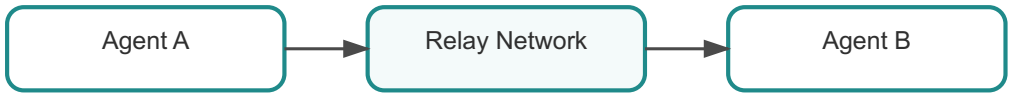


Figure 17.4. Federation resilience

ACP message identifiers and signature verification ensure that duplicate deliveries caused by retries can be safely detected by agents.

17.6 Trust Boundaries and Governance

Federation also introduces explicit **trust boundaries** between participating organizations.

Each organisation controls:

- which identities it trusts
- which relays are allowed to exchange messages
- which capabilities may cross organisational boundaries

The relay network, therefore, acts as a **policy enforcement layer** between autonomous systems.

This design reflects a broader trend in distributed architecture: security decisions are increasingly made at identity and policy layers rather than purely at network boundaries.

17.7 Federation and Ecosystem Growth

Federated relay networks enable ACP ecosystems to scale beyond a single organization.

Instead of requiring a centralised coordination platform, ACP allows independent infrastructures to cooperate through protocol interoperability.

Over time, this can produce large multi-organisation networks where autonomous agents collaborate across institutional boundaries.

Examples might include:

- supply chain coordination systems
- financial market infrastructure
- scientific research networks
- cross-organisational AI ecosystems

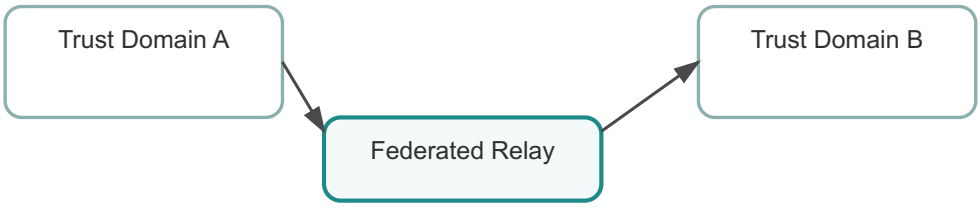


Figure 17.5. Ecosystem federation

This decentralised growth model mirrors the evolution of the Internet itself.

17.8 Operational Considerations

Operating federated relay networks introduces additional responsibilities for platform teams.

Operational concerns include:

- monitoring federation links
- auditing cross-organisation message flows
- managing trust relationships between relay operators
- coordinating incident response across domains

The ACP CLI and operational tooling described in earlier chapters provide mechanisms for inspecting relay status and federation routing behaviour.

These tools allow operators to diagnose problems without disrupting application logic.

17.9 Summary

Relay federation allows ACP networks to span multiple infrastructure domains while preserving local governance and security policies.

By separating routing infrastructure from message semantics, ACP enables organisations to collaborate without surrendering control over their internal systems.

This architectural model allows autonomous ecosystems to grow gradually through federated cooperation rather than centralised coordination.

In the next chapter, we explore **enterprise deployment patterns** and how ACP infrastructures are integrated into real operational environments.

Chapter 18 - Enterprise Deployment Patterns

Enterprise environments impose requirements that rarely appear in small development deployments. Security governance, regulatory compliance, operational reliability, and integration with existing infrastructure all influence architectural decisions.

ACP was designed with these constraints in mind, building on the layered design introduced earlier in the book.

18.1 Typical Enterprise Architecture

A typical enterprise ACP deployment resembles the following layered model. Each layer corresponds to infrastructure that enterprises already operate.

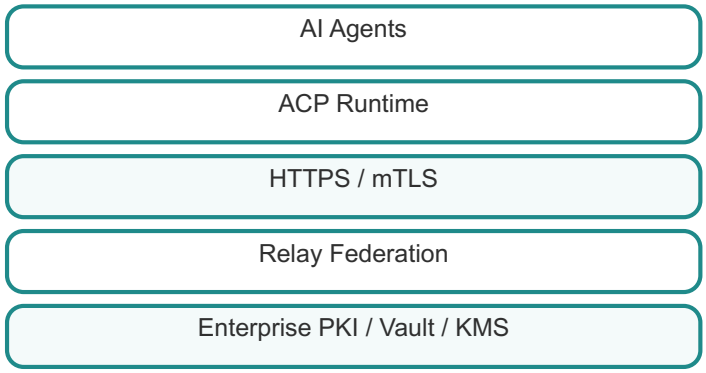


Figure 18.1. Enterprise Architectural Stack

The runtime enforces protocol semantics. Transport layers enforce secure network communication. Relay networks manage routing between environments. Enterprise PKI or secrets systems anchor cryptographic trust.

This alignment with existing infrastructure significantly lowers the barrier to adoption.

18.2 Integration with Existing Systems

Enterprise architects rarely deploy new protocols in isolation. ACP must coexist with API gateways, service meshes, identity providers, and logging systems already present in the environment.

Because ACP treats transport and key custody as separate layers, it can integrate with these systems rather than replacing them.

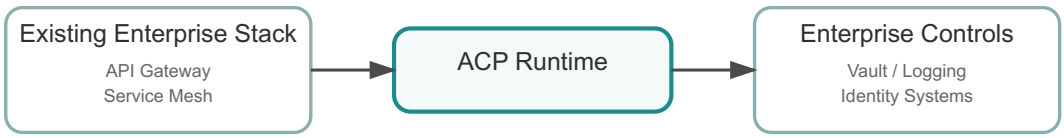


Figure 18.2. Enterprise Integration

For example, an ACP agent might run behind an existing API gateway while still using Vault for key management and a relay network for cross-domain routing.

18.3 Gradual Adoption

Enterprises typically begin by experimenting with ACP in limited contexts. Overlay wrappers allow existing services to participate without rewriting their architecture.

Over time organizations may introduce discovery registries and relay networks, gradually expanding the protocol's role.

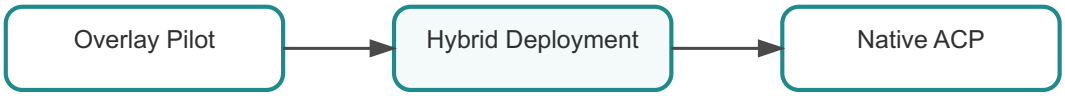


Figure 18.3. Enterprise Migration

This phased adoption model allows teams to evaluate ACP's benefits before committing to large architectural changes.

18.4 Summary

Enterprise deployment patterns reflect the realities of large organisations. Protocols must integrate with existing infrastructure and governance processes rather than requiring wholesale replacement.

ACP's layered architecture and overlay adoption model make this possible.

Chapter 19 - Cross-organisation Collaboration

Some of the most compelling applications of ACP involve collaboration between systems operated by different organisations. As introduced earlier in the book, autonomous systems increasingly interact across institutional boundaries , supply-chain platforms coordinating logistics, financial institutions exchanging market signals, and research systems sharing experimental results. These environments expose the limits of traditional integration approaches.

Conventional API integrations often assume stable relationships between a small number of services. Authentication schemes, endpoint addresses, and operational procedures are negotiated in advance and rarely change. That model works when interactions are limited and predictable. However, modern distributed ecosystems are neither. Organisations evolve independently, infrastructure changes frequently, and systems must collaborate dynamically without manual coordination for every interaction.

ACP was designed precisely for these conditions. By combining persistent identity, cryptographic message verification, transport independence, and relay federation, ACP provides a collaboration layer that can span multiple organisations while preserving governance and operational control.

19.1 The Cross-organisation Integration Challenge

To understand the importance of ACP's model, consider how organisations traditionally integrate systems.

A typical cross-organisation integration process includes:

- negotiating API contracts
- exchanging authentication credentials
- configuring firewall and network access
- maintaining endpoint mappings
- coordinating upgrades between teams

These steps often take weeks or months to complete. Once established, integrations are difficult to evolve because changes require coordination across multiple organisations.

Researchers studying distributed enterprise integration have repeatedly noted that tightly coupled system interfaces create long-term operational fragility. Martin Fowler and Gregor Hohpe describe this phenomenon in enterprise integration literature: systems that depend on rigid API contracts become brittle when organisations change independently.

Autonomous agents introduce even greater complexity. Agents may appear dynamically, support evolving capabilities, or interact with multiple organisations simultaneously. Static integration agreements simply cannot keep up with this level of dynamism.

ACP addresses this challenge by shifting collaboration from **endpoint agreements** to **identity-based interaction**.

19.2 Identity-First Collaboration

At the core of ACP's cross-organisation model is the identity system described in Chapter 3.

Each participant possesses a persistent identity anchored in cryptographic keys rather than infrastructure location. When two organisations interact through ACP, they establish trust relationships between identities rather than between network endpoints.

This distinction has significant implications.

Traditional integrations depend on infrastructure configuration:

```
Partner API Endpoint → Authentication Token → Service Interaction
```

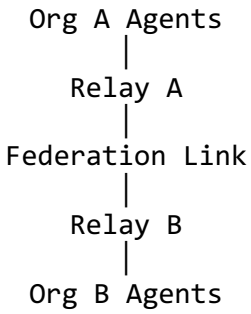
ACP interactions depend on identity verification:

```
Agent Identity → Signed Message → Capability Invocation
```

Because identities remain stable even when infrastructure changes, organisations can evolve their internal systems without renegotiating every external integration.

19.3 Federation as an organisational Boundary

Cross-organisation ACP collaboration typically occurs through **relay federation**, introduced in Chapter 17.



Each organisation maintains control of its internal infrastructure while participating in a shared protocol ecosystem.

The federation link acts as a controlled bridge between domains. Messages pass through the bridge while remaining cryptographically verifiable end-to-end.

This architecture allows organisations to collaborate without exposing internal network structures or requiring shared infrastructure ownership.

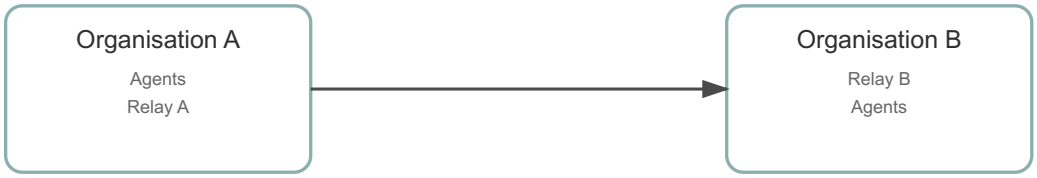


Figure 19.1. Cross-organisation federation

19.4 Governance Across organisations

Cross-organisation collaboration introduces governance challenges beyond technical connectivity.

organisations must determine:

- which external identities are trusted
- which capabilities may cross organisational boundaries
- which relay networks are permitted for communication
- how message flows are audited and monitored

ACP addresses these concerns through structured protocol metadata. Because messages contain explicit identity and capability information, governance policies can be applied at the protocol level.

For example, a relay may enforce rules such as:

- allow capability `trade_order` only from verified partner identities
- deny capabilities originating from unknown certificate authorities
- route specific capabilities through compliance monitoring infrastructure

These rules can be enforced without decrypting payload data, preserving confidentiality while maintaining administrative control.

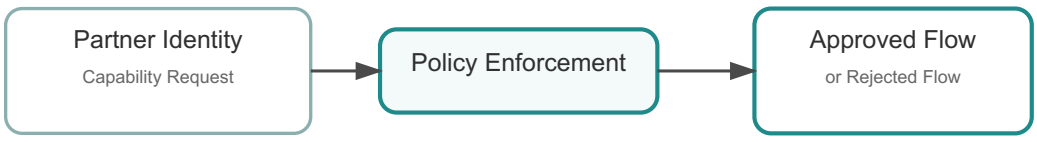


Figure 19.2. Policy enforcement

19.5 Trust Models in Federated Systems

Establishing trust between organisations is rarely binary. Instead, systems often rely on layered trust models.

Typical layers include:

1. Infrastructure trust (TLS and network controls)
2. Identity trust (cryptographic verification of agent identities)
3. Policy trust (organisational rules governing capabilities)

ACP integrates all three layers.

Transport security protects communication channels. Identity signatures verify the origin of messages. Governance policies enforce organisational rules regarding interaction patterns.

This layered trust model resembles modern **zero-trust security architectures**, which assume that network boundaries alone cannot guarantee system integrity.

19.6 Operational Challenges in Cross-organisation Systems

Operating cross-organisation collaboration networks introduces unique operational challenges.

Organisations must coordinate monitoring and incident response across domains. Message routing paths may span multiple relay networks, making observability more complex. Additionally, governance policies may change independently in each organisation.

ACP mitigates these challenges through structured metadata and standardised protocol semantics.

Message identifiers allow tracing interactions across organisations. Identity metadata allows security teams to identify the source of interactions even when messages pass through multiple intermediaries. Capability identifiers provide semantic context that helps operators understand system behaviour.

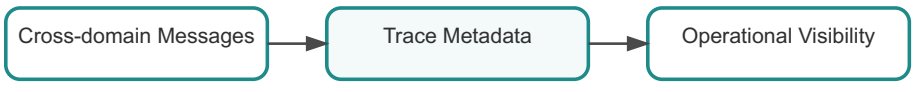


Figure 19.3. Operational visibility

19.7 Example: Supply Chain Coordination

To illustrate how ACP enables cross-organisation collaboration, consider a supply-chain scenario.

A manufacturer coordinates with multiple suppliers, logistics providers, and distributors. Each participant operates independent systems and infrastructure. Traditional integration would require maintaining dozens of API integrations.

With ACP, each participant exposes capabilities such as:

- `inventory_status`
- `shipment_update`
- `delivery_confirmation`

Messages are exchanged between identities rather than between endpoints. Relay federation routes the messages across organisational boundaries while maintaining policy enforcement.

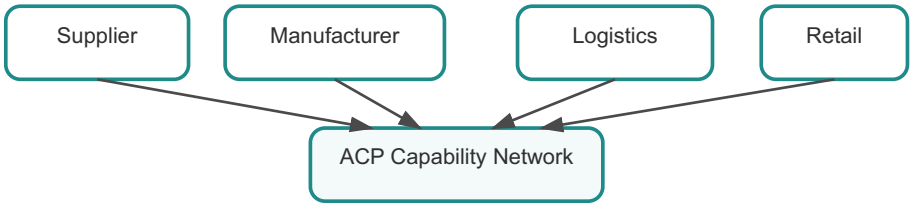


Figure 19.4. Supply chain ecosystem

This architecture allows the ecosystem to evolve dynamically as new participants join the network.

19.8 Lessons from Internet-Scale Systems

The architecture of ACP federation mirrors lessons learned from earlier large-scale systems, such as email and Internet routing.

Email systems do not require every organisation to maintain direct connectivity with every other organisation. Instead, mail transfer agents form a federated routing network.

Similarly, the Internet itself is composed of independently operated networks that exchange traffic through well-defined protocols while maintaining local autonomy.

ACP applies these principles to machine collaboration.

Instead of building centralised integration platforms, ACP allows autonomous systems to participate in a federated ecosystem governed by shared protocol semantics.

19.9 Summary

Cross-organisation collaboration is one of the most challenging aspects of modern distributed systems. Traditional integration approaches struggle when systems evolve independently or when large numbers of participants interact dynamically.

ACP addresses these challenges through identity-based interaction, relay federation, and protocol-level governance mechanisms. Organisations retain control of their infrastructure while participating in a shared collaboration ecosystem.

By separating identity from infrastructure and embedding trust into message semantics, ACP enables autonomous systems to collaborate across organisational boundaries without tightly coupled integrations.

Conclusion

Reframing the Problem

At the beginning of this book, we introduced what was described as the *machine collaboration problem*. The problem was not presented as a limitation of a specific tool, framework, or technology, but as a structural gap in how distributed systems have evolved.

Modern systems are highly capable of connecting. They are less able to sustain interaction.

This distinction has been a recurring theme throughout the book. It is worth restating it here, not as repetition, but as consolidation.

Systems do not fail because they cannot communicate. They fail because communication is not sufficient to maintain coordination over time.

As systems evolve, assumptions embedded at the time of integration begin to break. Endpoints change, infrastructures shift, trust boundaries expand, and participants evolve independently. What was once a stable connection becomes a fragile dependency.

The problem is not visible when systems are built. It becomes visible when systems change.

ACP is not a response to communication. It is a response to change.

From Integration to Interaction

Traditional distributed systems have been designed around integration. Integration concerns establishing a connection between two systems. It is typically solved through:

- defining an endpoint
- agreeing on a contract
- exchanging messages

This model works well when systems are stable and interactions are predictable.

Autonomous systems operate differently. They are dynamic, distributed, and often independent. They require interaction rather than integration.

Interaction implies:

- continuity of identity
- stability of semantics
- independence from infrastructure
- the ability to adapt as participants evolve

Throughout Part I, we introduced the concepts required to support interaction: agents, identity, messages, capabilities, discovery, and transport independence. These concepts were not defined arbitrarily. Each exists because integration-based models fail to provide the guarantees required for interaction.

The transition from integration to interaction is not merely technical. It is architectural.

The Missing Layer Revisited

In the Introduction, we described ACP as addressing a missing layer in distributed systems.

This layer sits between application logic and transport infrastructure. It defines how autonomous participants interact as independent entities.

By the time we reach this conclusion, the nature of that layer should be clear.

It consists of:

- persistent identity, independent of infrastructure
- message-level trust, independent of transport
- dynamic discovery, independent of static configuration
- transport abstraction, allowing delivery mechanisms to evolve
- relay mechanisms that support communication across boundaries

Individually, none of these ideas is entirely new. Identity systems, message signing, service discovery, and transport abstraction have all been explored in different contexts.

What ACP provides is a coherent model that brings these ideas together into a single protocol layer.

This is the key distinction.

ACP is not a collection of features. It is a model of interaction.

Identity as the Anchor of Trust

One of the most significant architectural shifts introduced by ACP is treating identity as a first-class protocol primitive.

In many systems, identity is derived from infrastructure. A service is identified by its location. Trust is inferred from network boundaries or transport-level security.

As systems evolve, these assumptions become difficult to maintain.

By separating identity from endpoint, ACP allows trust relationships to persist even as infrastructure changes. This is not simply a security improvement. It is a stability mechanism.

Identity becomes the anchor around which interactions can evolve.

As explored in Chapter 3, this approach aligns with broader trends in distributed systems, where identity-based models are replacing location-based trust.

Messages as Self-Contained Interactions

The ACP message model extends this principle by embedding identity, semantics, and trust directly into the message envelope.

In traditional systems, meaning is often inferred from context:

- the endpoint that was called
- the topic to which a message was published
- the application logic interpreting the payload

ACP makes this meaning explicit.

A message carries:

- who sent it
- what it represents
- how it can be verified

This allows messages to move across transports, intermediaries, and organisational boundaries without losing their semantics.

As discussed in Chapter 4, this property is essential for building systems that can evolve without breaking interaction.

Discovery as the Bridge Between Identity and Infrastructure

Discovery connects the stable world of identity with the dynamic world of infrastructure.

Without discovery, systems must rely on manual configuration. This approach does not scale in environments with numerous, constantly changing participants.

ACP introduces discovery as a protocol concern, allowing systems to locate one another dynamically.

The well-known endpoint, explored in Chapter 5, provides a simple bootstrap mechanism. More advanced discovery approaches enable governance and control in enterprise environments.

Discovery is not merely a convenience. It is the mechanism that allows identity to remain stable while infrastructure evolves.

Transport as an Implementation Detail

One of the most subtle yet important design decisions in ACP is treating transport as an implementation detail.

In many systems, transport defines behaviour. HTTP endpoints encode semantics. Messaging systems impose delivery guarantees that applications depend on.

ACP reverses this relationship.

Transport becomes responsible for delivery, not meaning.

As explored in Chapter 6, this allows systems to evolve their communication mechanisms without redefining their interactions. It also allows governance and trust to operate independently of infrastructure choices.

This separation is essential for long-term system stability.

Security as a Property of Interaction

Security in ACP is not confined to the transport layer. It is embedded in the interaction itself.

By combining identity, signatures, and optional encryption, ACP ensures that trust is maintained even when messages pass through untrusted infrastructure.

As discussed in Chapter 7, this allows systems to operate across complex environments while preserving consistent security guarantees.

This approach also supports governance requirements, enabling organisations to reason about interactions independently of network topology.

Relays and Boundaries

Relays extend the protocol into environments where direct communication is not possible.

They provide a mechanism for routing messages across network and organisational boundaries without interpreting their contents.

As explored in Chapter 8, relays are not centralised controllers. They are intermediaries that preserve protocol semantics while enabling connectivity.

This distinction is important. It allows ACP to support federated ecosystems where organisations retain control over their own systems while still participating in shared interactions.

Capability and Evolution

Capability negotiation, introduced in Chapter 9, addresses another fundamental requirement: systems evolve independently.

Without negotiation, every change becomes a breaking change. With negotiation, systems can adapt to one another.

This supports incremental evolution, allowing ecosystems to grow without requiring synchronised updates across all participants.

From Architecture to Usage

In Part III, we moved from architecture to usage.

We explored how ACP can be applied in practice:

- building agents
- using overlay models
- interacting through SDKs and CLI tools
- demonstrating interoperability through multi-agent examples

These chapters illustrate that ACP is not purely theoretical. It can be applied incrementally, starting from existing systems and evolving towards native deployments.

The overlay model demonstrates how ACP can be introduced without requiring a complete architectural rewrite.

Enterprise and Governance Considerations

Part IV examined the implications of ACP in enterprise environments.

As systems scale and cross organisational boundaries, governance becomes critical. Identity, policy enforcement, auditability, and operational control must all be considered.

ACP does not replace these concerns. It provides a protocol layer that makes them easier to express and enforce.

By separating identity, trust, and transport, ACP allows organisations to apply governance consistently across interactions.

This becomes increasingly important in regulated industries, where traceability and control are essential.

What ACP Is Revisited

By this point, it should be clear that ACP is not:

- a messaging system

- a framework
- a transport protocol

It is a **collaboration protocol**.

Its purpose is to define how autonomous participants interact in a way that remains stable over time.

This distinction is subtle but important. It explains why ACP is designed as a protocol rather than a tool.

Tools solve local problems. Protocols define shared behaviour.

ACP operates at the system level, not the component level.

Where ACP Fits

Not every system requires ACP.

Small, tightly coupled systems may function perfectly well with traditional integration approaches. Systems that operate within a single, stable environment may not encounter the challenges described in this book.

ACP becomes relevant when:

- systems are distributed across environments
- participants evolve independently
- interactions must persist over time
- governance and trust requirements are significant

In such environments, the absence of a collaboration layer becomes a limiting factor.

ACP provides an effective, simple way to address that limitation.

Adoption as an Evolution

ACP is designed to be adopted incrementally.

Organisations can begin by introducing ACP as an overlay on existing systems. This allows them to experiment with identity, discovery, and message semantics without changing their underlying architecture.

Over time, systems can move towards native ACP deployments, where the protocol becomes the primary interaction layer.

This progression reflects the book’s broader theme: systems evolve.

ACP does not require a single transformation. It supports gradual adoption.

Looking Forward

The rise of autonomous systems is still in its early stages.

As systems become more capable, more distributed, and more independent, the need for reliable collaboration mechanisms will increase.

Multi-agent systems, AI-driven workflows, and cross-organisational ecosystems all depend on systems' ability to interact safely and consistently.

The ideas presented in this book are not tied to a specific technology or moment in time. They reflect a broader shift in how distributed systems are designed.

Whether ACP becomes the dominant model is not the most important question.

The more important question is whether the problem it addresses is real.

Final Reflection

Throughout this book, we have examined a set of recurring themes:

- systems evolve over time
- assumptions break under change
- interaction requires more than communication
- trust must be explicit
- governance must be enforceable

ACP brings these ideas together into a single protocol model.

The question is not whether ACP is correct.

The question is whether the challenges it addresses are present in your systems.

If they are, then the concepts explored in this book provide a framework for thinking about them, and for building systems that remain stable as they evolve.

If they are not, then ACP may not be necessary.

In either case, understanding the problem is the first step.

Glossary

This glossary defines the key terms used throughout *Designing Autonomous Systems with ACP*.
The goal is to provide quick reference definitions so readers can easily re-anchor their understanding when encountering unfamiliar terminology.

Agent

An autonomous participant in an ACP network. An agent is capable of sending and receiving ACP messages, exposing capabilities, and collaborating with other agents.

Agents may represent AI systems, automation services, orchestration components, or other autonomous software actors.

Identity

A persistent identifier representing an ACP agent. Identity is independent of infrastructure location.

Example:

```
agent:ricardo.chess@demo
```

Identity is used to establish trust relationships and verify message authenticity.

Identity Document

A signed document describing an agent's public cryptographic material and metadata.

Identity documents typically include:

- public signing key
- encryption key
- agent identifier
- optional metadata

They allow other agents to verify message signatures.

Endpoint

A network location where an agent can currently be reached.

Examples include:

```
https://agent.example.com/acp
amqp://broker.example.com/agent.queue
```

Unlike identity, endpoints may change as infrastructure evolves.

Capability

A semantic action that an agent can perform or request through an ACP message.

Examples:

- `ping`
- `execute_workflow`
- `challenge`
- `move`

Capabilities allow agents to negotiate interactions and evolve independently.

ACP Message

The fundamental communication unit in the protocol. An ACP message consists of an envelope containing metadata, signatures, and optional encryption along with the application payload.

Simplified envelope:

```
ACP Envelope
 ├ message_id
 ├ agent_id
 ├ capability
 ├ timestamp
 ├ signature
 └ encrypted payload
```

Message Identifier

A unique identifier attached to each ACP message. It enables message tracing, deduplication, and correlation across distributed systems.

Discovery

The process of locating agents in the network by resolving identities into communication hints such as endpoints or transport information.

Discovery mechanisms include:

- static configuration
- discovery registries
- well-known endpoints

Well-Known Endpoint

A bootstrap discovery mechanism defined by the path:

```
/.well-known/acp
```

Agents expose this endpoint to publish metadata describing their identity and transport hints.

Transport

The underlying communication channel used to deliver ACP messages.

Common transports include:

- HTTP(S)
- AMQP
- MQTT
- relay networks

ACP is designed to be transport-independent.

Transport Binding

A specification describing how ACP messages are carried over a specific transport.

For example:

- HTTP binding
- AMQP binding
- MQTT binding

Transport bindings preserve the ACP envelope without changing protocol semantics.

Relay

An intermediary service that forwards ACP messages between agents.

Relays allow communication when direct connectivity is not possible or when routing policies must be enforced.

Example routing path:

```
Agent A → Relay → Agent B
```

Relay Federation

A network architecture where multiple relays cooperate to route messages between different organisations or network domains.

Federation enables cross-domain communication while maintaining governance boundaries.

Overlay Mode

A deployment model in which ACP semantics are layered on top of existing services.

Existing APIs continue to operate normally while ACP envelopes wrap the communication.

Example:

```
Existing Service + ACP Wrapper
```

Overlay mode allows incremental adoption.

Native ACP

A deployment model where ACP becomes the primary communication layer between agents.

Native deployments typically include:

- ACP runtimes
- discovery infrastructure
- relay networks

Discovery Registry

A service that maps agent identities to current communication endpoints.

Registries provide centralised governance and operational visibility in large deployments.

Runtime

The component that implements the ACP protocol inside an SDK.

The runtime handles:

- message construction
- signature verification
- payload encryption/decryption
- discovery resolution
- transport interaction

SDK (Software Development Kit)

A language-specific implementation of the ACP runtime used by developers to build agents.

Current implementations include:

- Python
- Java
- Rust
- TypeScript
- Go
- Mojo

PKI (Public Key Infrastructure)

A system used to manage certificates and cryptographic trust relationships.

Enterprises often integrate ACP identity with existing PKI systems.

Vault / KMS

Secret management systems used to store cryptographic keys securely.

ACP can retrieve signing and encryption keys from these systems through key provider abstractions.

Federation

A collaboration model where multiple organisations operate independent ACP infrastructure while allowing controlled communication between them.

Federation typically relies on relay networks and shared trust policies.

Governance

The policies and controls that regulate how agents communicate.

Governance mechanisms may include:

- trusted identity lists
- capability restrictions
- routing policies
- audit logging

These mechanisms are especially important in regulated environments.

INDEX